

TRƯỜNG

KHOA CÔNG NGHỆ THÔNG TIN



[LOGO TRƯỜNG]

ĐỒ ÁN MÔN HỌC

PHÁT TRIỂN ỨNG DỤNG DI ĐỘNG

Đề tài:

XÂY DỰNG ỨNG DỤNG PETSHOP CỬA HÀNG THÚ CƯNG VÀ THỨC ĂN CHO THÚ CƯNG TRÊN NỀN TẢNG ANDROID

Giảng viên hướng dẫn :

Nhóm sinh viên thực hiện (Nhóm):

1. Nguyễn Hữu Đức Thọ – MSSV:
2. Đào Trúc Mai – MSSV:
3. Nguyễn Văn Trường – MSSV:
4. – MSSV:

Năm học 2025 – 2026

TRƯỜNG

KHOA CÔNG NGHỆ THÔNG TIN



ĐỒ ÁN MÔN HỌC

PHÁT TRIỂN ỨNG DỤNG DI ĐỘNG

XÂY DỰNG ỨNG DỤNG PETSHOP – CỬA HÀNG THÚ CƯNG

VÀ THỨC ĂN CHO THÚ CƯNG TRÊN NỀN TẢNG ANDROID

Tên đề tài	Ứng dụng PetShop
Loại đề tài	Đồ án môn học – Phát triển ứng dụng di động
Nền tảng	Android (Java, minSdk 24, targetSdk 36)
Công nghệ chính	Firebase, VNPay, OpenAI, MVVM + Repository
Giảng viên hướng dẫn	
Mã lớp / Học kỳ	 / Học kỳ năm học 2025-2026
Thời gian thực hiện	/...../2026 →/...../2026
Nhóm thực hiện	Nhóm – 04 thành viên

DANH SÁCH NHÓM

STT	Họ và tên	MSSV	Vai trò trong nhóm
1	Nguyễn Hữu Đức Thọ		Trưởng nhóm – TV1

2	Đào Trúc Mai		Thành viên – TV2
3	Nguyễn Văn Trường		Thành viên – TV3
4			Thành viên – TV4
			

TP., Tháng năm 2026

LỜI CẢM ƠN

Lời đầu tiên, nhóm thực hiện đề tài xin gửi lời cảm ơn chân thành và sâu sắc nhất đến quý Thầy, Cô Khoa Công nghệ Thông tin – Trường đã tận tình giảng dạy và truyền đạt cho chúng em những kiến thức nền tảng vô cùng quý báu trong suốt thời gian học tập tại trường. Những kiến thức đó là hành trang, là nền tảng vững chắc giúp nhóm có thể tiếp cận và hoàn thành đề tài này.

Đặc biệt, nhóm xin được gửi lời cảm ơn sâu sắc đến giảng viên hướng dẫn – – người đã trực tiếp định hướng đề tài, dành nhiều thời gian, công sức để hướng dẫn, đóng góp ý kiến và chỉnh sửa cho từng phần nội dung. Sự tận tâm, nhiệt tình cùng những góp ý quý báu của Thầy/Cô là động lực rất lớn giúp nhóm vượt qua được các khó khăn về kiến thức, công nghệ và tổ chức nhóm trong quá trình thực hiện.

Nhóm cũng xin gửi lời cảm ơn đến các bạn cùng lớp đã nhiệt tình hỗ trợ kiểm thử ứng dụng, đóng góp ý kiến phản hồi từ góc nhìn người dùng để sản phẩm cuối cùng hoàn thiện hơn. Đồng thời, nhóm xin cảm ơn cộng đồng lập trình Android trên các diễn đàn StackOverflow, Android Developers, Firebase Documentation, VNPay Documentation, OpenAI API Reference... đã chia sẻ tri thức một cách cởi mở để chúng em có nguồn tham khảo phong phú trong quá trình triển khai.

Mặc dù đã cố gắng và nỗ lực hết sức, nhưng do thời gian thực hiện có hạn cùng với kinh nghiệm thực tế chưa nhiều, đề tài chắc chắn không tránh khỏi những thiếu sót. Nhóm kính mong nhận được sự góp ý chân thành từ quý Thầy, Cô và các bạn để đề tài được hoàn thiện hơn nữa, đồng thời giúp các thành viên trong nhóm rút kinh nghiệm cho các dự án trong tương lai.

Một lần nữa, nhóm xin trân trọng cảm ơn!

TP., Tháng năm 2026

Nhóm sinh viên thực hiện

(Ký và ghi rõ họ tên)

LỜI CAM ĐOAN

Nhóm thực hiện đề tài xin cam đoan rằng đề tài “Xây dựng ứng dụng PetShop – Cửa hàng thú cưng và thức ăn cho thú cưng trên nền tảng Android” là công trình nghiên cứu, phân tích, thiết kế và lập trình do chính nhóm thực hiện dưới sự hướng dẫn của giảng viên

Toàn bộ mã nguồn được nhóm trực tiếp viết bằng ngôn ngữ Java trên Android Studio, có tham khảo tài liệu chính thức của Android Developers, Firebase, VNPay và OpenAI; các đoạn mã tham khảo từ bên thứ ba (nếu có) đều được trích dẫn rõ ràng trong phần Tài liệu tham khảo. Các tài liệu, hình ảnh, sơ đồ, bảng biểu được sử dụng trong báo cáo đều có nguồn gốc minh bạch hoặc do nhóm tự thiết kế.

Nhóm cam kết không sao chép báo cáo từ bất kỳ đề tài, khóa luận hay tài liệu nào khác. Mọi nội dung, kết quả nêu trong báo cáo đều trung thực, phản ánh đúng quá trình làm việc và sản phẩm thực tế. Nếu phát hiện có bất kỳ sự gian lận, sao chép trái phép nào, nhóm xin hoàn toàn chịu trách nhiệm trước Hội đồng đánh giá và Khoa Công nghệ Thông tin.

TP., Tháng năm 2026

Đại diện nhóm thực hiện

(Ký và ghi rõ họ tên)

Nguyễn Hữu Đức Thọ

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN

Họ và tên giảng viên hướng dẫn :

.....
.....

Đơn vị công tác :

.....
.....

Tên đề tài : Xây dựng ứng dụng PetShop – Cửa hàng thú cưng và thức ăn cho thú cưng trên nền tảng Android.

Nhận xét chung về tinh thần, thái độ làm việc của nhóm:

.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....

Nhận xét về nội dung và kết quả thực hiện đề tài:

.....
.....
.....
.....
.....
.....
.....
.....

.....
.....
.....
.....
.....
.....
.....
.....

Đánh giá chung:

.....
.....
.....
.....
.....
.....

Điểm đề xuất (thang 10):/10

TP., Tháng năm 2026

Giảng viên hướng dẫn

(Ký và ghi rõ họ tên)

BẢNG PHÂN CÔNG CÔNG VIỆC NHÓM

Đề tài được thực hiện bởi nhóm 04 sinh viên với sự phân công rõ ràng theo bốn module chức năng chính của ứng dụng. Việc chia module bám sát kiến trúc MVVM của dự án (Model – View – ViewModel) và đảm bảo mỗi thành viên phụ trách một mảng nghiệp vụ trọn vẹn, từ thiết kế CSDL đến giao diện và xử lý nghiệp vụ. Bảng dưới đây mô tả khái quát phần việc của mỗi thành viên; chi tiết tiến độ và sản phẩm bàn giao xem tại Phụ lục A.

Bảng 1: Phân công công việc tổng quát giữa 04 thành viên

TV	Họ và tên	Module phụ trách	Phân công việc chính	Tỉ lệ
TV1	Nguyễn Hữu Đức Thọ (Trưởng nhóm)	Tài khoản – Hồ sơ (Chương 3)	Phân tích & thiết kế Use Case Auth; Firebase Authentication (Email/Password, Google Sign-In, thiết kế Facebook Login); OTP qua JavaMail; Splash; Login/Register; Profile & Edit Profile; Quản lý địa chỉ; Hệ thống Notification; SessionManager / SharedPreferences; tổng hợp báo cáo & họp nhóm.	25 %
TV2	Đào Trúc Mai	Danh mục & Trang chủ (Chương 4)	Thiết kế Model Pet/Food/CATEGORY/Banner/Review ; Repository Pet/Food/CATEGORY; HomeFragment (banner ViewPager2, lời chào, tìm kiếm, lọc theo danh mục); CategoryFragment; PetDetailActivity &	25 %

			FoodDetailActivity; tích hợp Glide + Firebase Storage hiển thị ảnh; chức năng tìm kiếm và lọc sản phẩm.	
TV3	Nguyễn Văn Trường	Giỏ hàng & Thanh toán (Chương 5)	Thiết kế Model Cart/CartItem/Order/OrderItem/ReturnRequest/PaymentTransaction; CartRepository; CartFragment & CartActivity; CheckoutActivity; tính phí ship theo vùng (ShippingHelper); áp voucher/promotion; tích hợp VNPay (VNPayHelper – HMAC-SHA512, VNPayWebView Activity, VNPayResultActivity Deep Link); OrderHistory & OrderDetail; ReturnRequestActivity.	25 %
TV4		Quản trị & Chatbot (Chương 6)	Phân quyền Admin (AdminSetupHelper); AdminActivity dashboard real-time (AdminViewModel + SnapshotListener); Quản lý User / Category / Pet / Food / Promotion / Voucher / Order / Return; tải ảnh sản phẩm	25 %

			(StorageHelper); thiết kế hội thoại Chatbot; ChatViewModel gọi OpenAI Chat Completions bằng OkHttp + Gson; lưu lịch sử chat trên Firestore.	
--	--	--	--	--

Ngoài phần module riêng, cả 04 thành viên cùng tham gia các nhiệm vụ chung gồm: (1) phân tích yêu cầu, vẽ Use Case tổng quát (Chương 1, 2); (2) thiết kế kiến trúc tổng thể, mô hình dữ liệu Firestore và Security Rules (Chương 2); (3) kiểm thử chéo các module (Chương 7); (4) chuẩn bị slide trình bày, viết báo cáo, demo trực tiếp.

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Nguyên gốc / Tiếng Anh	Giải thích
API	Application Programming Interface	Giao diện lập trình ứng dụng
APK	Android Package Kit	Định dạng cài đặt ứng dụng Android
BaaS	Backend as a Service	Dịch vụ backend dạng nền tảng (vd: Firebase)
CRUD	Create – Read – Update – Delete	Bốn tác vụ cơ bản với dữ liệu
COD	Cash On Delivery	Thanh toán khi nhận hàng
FCM	Firebase Cloud Messaging	Dịch vụ push notification của Google
HMAC	Hash-based Message Authentication Code	Mã xác thực thông điệp dựa trên hàm băm
JSON	JavaScript Object Notation	Định dạng trao đổi dữ liệu phổ biến
LiveData	—	Lớp dữ liệu nhận thức vòng đời của AndroidX
MVC / MVVM	Model-View-Controller / Model-View-ViewModel	Các mẫu kiến trúc phần mềm
NoSQL	Not only SQL	CSDL phi quan hệ (Firestore là một ví dụ)
OAuth	Open Authorization	Chuẩn xác thực mở (Google/Facebook Sign-In)
OTP	One-Time Password	Mật khẩu sử dụng một lần (gửi qua email)
REST	Representational State Transfer	Kiểu kiến trúc dịch vụ web phổ biến
SDK	Software Development Kit	Bộ công cụ phát triển phần mềm
SHA-512	Secure Hash Algorithm 512-bit	Hàm băm 512 bit dùng cho HMAC VNPay
SMTP	Simple Mail Transfer Protocol	Giao thức gửi email
UI / UX	User Interface / User Experience	Giao diện / Trải nghiệm người dùng
URL	Uniform Resource Locator	Định danh tài nguyên trên Internet
VNPAY	—	Cổng thanh toán điện tử Việt Nam
XML	eXtensible Markup Language	Định dạng đánh dấu mở rộng (Android layout)

DANH MỤC HÌNH ẢNH

(Tự động cập nhật sau khi nhấn F9 trong Word)

Mục lục hình ảnh sẽ được sinh tự động bằng cách: trong Word vào References → Insert Table of Figures → chọn Caption label = “Hình”.

- Hình 1.1. Sơ đồ Use Case tổng quát của ứng dụng PetShop
- Hình 1.2. Sơ đồ kiến trúc MVVM + Repository + Firebase
- Hình 1.3. Mô hình tổ chức package mã nguồn
- Hình 2.1. Sơ đồ ERD logic của các collection trên Firestore
- Hình 2.2. Sơ đồ collection Firestore của ứng dụng PetShop
- Hình 2.3. Tóm tắt logic Firestore Security Rules
- Hình 2.4. Sơ đồ luồng dữ liệu giữa các tầng MVVM
- Hình 2.5. Sơ đồ trạng thái đơn hàng (State Diagram)
- Hình 2.6. Bộ màu chủ đạo và icon của ứng dụng
- Hình 3.1. Activity Diagram – luồng đăng ký, đăng nhập
- Hình 3.2. Hành trình người dùng (Customer Journey)
- Hình 4.1. Giao diện màn hình Trang chủ
- Hình 5.1. Activity Diagram – luồng đặt hàng & thanh toán
- Hình 5.2. Sequence Diagram – thanh toán qua VNPay
- Hình 5.3. Sơ đồ luồng dữ liệu thanh toán VNPay
- Hình 5.4. Sơ đồ trạng thái thanh toán hiển thị cho khách
- Hình 5.5. Activity Diagram – yêu cầu trả hàng
- Hình 6.1. Hành trình quản trị viên (Admin Journey)
- Hình 6.2. Sequence Diagram – Admin Dashboard real-time
- Hình 6.3. Sơ đồ kiến trúc Chatbot RAG nhẹ

DANH MỤC BẢNG BIỂU

Mục lục bảng biểu sẽ được sinh tự động bằng *References* → *Insert Table of Figures* → chọn *Caption label* = “Bảng”.

Bảng 0. Phân công công việc tổng quát giữa 04 thành viên

Bảng 1.1. So sánh các giải pháp Pet shop hiện có trên thị trường

Bảng 1.2. Yêu cầu chức năng (FR) cho khách hàng

Bảng 1.3. Yêu cầu chức năng (FR) cho quản trị viên

Bảng 1.4. Yêu cầu phi chức năng (NFR)

Bảng 2.1. Mô tả Use Case 'Đặt hàng và thanh toán'

Bảng 2.2. Mô tả Use Case 'Yêu cầu trả hàng / hoàn tiền'

Bảng 2.3. Cấu trúc các collection chính trên Firestore

Bảng 3.1. Các thuộc tính của Model User

Bảng 4.1. Các thuộc tính của Model Pet

Bảng 4.2. Các thuộc tính của Model Food

Bảng 5.1. Bảng giá phí vận chuyển theo vùng

Bảng 5.2. Tham số chính của VNPay buildPaymentUrl

Bảng 6.1. Các tab quản trị trong AdminActivity

Bảng 7.1. Test case module Tài khoản

Bảng 7.2. Test case module Catalog

Bảng 7.3. Test case module Order & Payment

Bảng 7.4. Test case module Admin

Bảng 7.5. Test case module Chatbot

MỤC LỤC

Mục lục được sinh tự động: trong Word vào References → Table of Contents → chọn “Automatic Table 1”. Sau khi nhấn Update Field (F9), Word sẽ liệt kê đầy đủ các chương, mục theo Heading 1/2/3 đã được định nghĩa trong báo cáo này.

MỞ ĐẦU

1. Lý do chọn đề tài
2. Mục tiêu của đề tài
3. Đối tượng và phạm vi nghiên cứu
4. Phương pháp thực hiện
5. Bố cục báo cáo

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI VÀ CÔNG NGHỆ SỬ DỤNG

- 1.1. Khảo sát hiện trạng các ứng dụng pet shop
- 1.2. Mô tả bài toán và yêu cầu chức năng
- 1.3. Công nghệ sử dụng
- 1.4. Công cụ phát triển và quản lý dự án

CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

- 2.1. Sơ đồ Use Case tổng quát
- 2.2. Đặc tả các Use Case chính
- 2.3. Sơ đồ hoạt động (Activity Diagram)
- 2.4. Thiết kế cơ sở dữ liệu trên Cloud Firestore
- 2.5. Kiến trúc tổng thể của ứng dụng
- 2.6. Thiết kế giao diện (UI/UX)

CHƯƠNG 3. XÂY DỰNG MODULE TÀI KHOẢN VÀ HỒ SƠ NGƯỜI DÙNG

CHƯƠNG 4. XÂY DỰNG MODULE DANH MỤC SẢN PHẨM VÀ TRANG CHỦ

CHƯƠNG 5. XÂY DỰNG MODULE GIỎ HÀNG, THANH TOÁN VÀ ĐƠN HÀNG

CHƯƠNG 6. XÂY DỰNG MODULE QUẢN TRỊ VÀ CHATBOT TƯ VẤN

CHƯƠNG 7. KIỂM THỬ, TRIỂN KHAI VÀ ĐÁNH GIÁ

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

TÀI LIỆU THAM KHẢO

PHỤ LỤC A. Bảng phân công công việc chi tiết và tiến độ

PHỤ LỤC B. Cấu trúc thư mục mã nguồn

PHỤ LỤC C. Sơ đồ Firestore và Security Rules

PHỤ LỤC D. Ảnh chụp màn hình toàn bộ ứng dụng

MỞ ĐẦU

1. Lý do chọn đề tài

Trong những năm gần đây, song song với sự phát triển nhanh chóng của nền kinh tế và mức sống ngày càng được nâng cao, nhu cầu nuôi thú cưng tại Việt Nam, đặc biệt ở các đô thị lớn, đã có sự gia tăng rõ rệt. Theo nhiều khảo sát thị trường, có khoảng 35–40% hộ gia đình thành thị đang nuôi ít nhất một thú cưng (chó, mèo, cá, chim, hamster, ...). Kéo theo đó là một thị trường khổng lồ về thú cưng giống, thức ăn chuyên biệt, phụ kiện chăm sóc, dịch vụ thú y, làm đẹp, gửi thú... với tốc độ tăng trưởng hai con số mỗi năm.

Tuy nhiên, ở thị trường trong nước, đa số giao dịch mua bán thú cưng và thức ăn cho thú cưng vẫn diễn ra theo các kênh truyền thống: cửa hàng vật lý, các bài đăng tự phát trên mạng xã hội (Facebook, Zalo, Chợ Tốt). Người dùng gặp một số khó khăn điển hình: (1) khó tìm đúng giống, đúng độ tuổi, đúng nguồn gốc thú cưng phù hợp với gia đình; (2) khó so sánh giá và xác minh chất lượng thức ăn theo loài, độ tuổi và cân nặng; (3) thiếu cơ chế bảo vệ giao dịch, không có lịch sử đơn hàng, không có quy trình hoàn trả/hoàn tiền minh bạch; (4) thiếu kênh tư vấn 24/7 cho người mới nuôi thú lần đầu.

Trong khi đó, các nền tảng thương mại điện tử lớn như Shopee, Lazada, Tiki tuy có bán đồ thú cưng nhưng không chuyên biệt, không tối ưu trải nghiệm tìm kiếm theo loài/giống/tuổi, không hỗ trợ đăng tin bán thú cưng theo từng cá thể (mỗi pet là một sản phẩm độc nhất với tình trạng AVAILABLE/RESERVED/SOLD), và không có chatbot tư vấn chuyên sâu về thú cưng.

Xuất phát từ thực tiễn đó, nhóm chọn đề tài “Xây dựng ứng dụng PetShop trên nền tảng Android” nhằm đề xuất một giải pháp ứng dụng di động chuyên biệt cho lĩnh vực thú cưng. Ứng dụng vừa giải quyết các bài toán nghiệp vụ của một website thương mại điện tử thông thường (đăng nhập, sản phẩm, giỏ hàng, thanh toán, đơn hàng), vừa bổ sung các tính năng đặc thù của ngành thú cưng (mỗi pet là một cá thể duy nhất, theo dõi tình trạng đặt giữ, phân loại thức ăn theo loài và lứa tuổi, chatbot tư vấn). Đề tài cũng là cơ hội để các thành viên vận dụng tổng hợp kiến thức về kiến trúc MVVM,

Firebase, OAuth 2.0 (Google Sign-In), HMAC-SHA512 (VNPay), tích hợp REST API bên ngoài (OpenAI), thiết kế UI/UX theo Material Design.

2. Mục tiêu của đề tài

Đề tài hướng tới các mục tiêu cụ thể sau:

1. Xây dựng được một ứng dụng Android hoàn chỉnh cho hai vai trò người dùng – Khách hàng (Customer) và Quản trị viên (Admin) – với điều hướng tách biệt rõ ràng.
2. Quản lý được hai loại sản phẩm đặc thù của ngành: thú cưng (Pet, mỗi pet là một cá thể) và thức ăn cho thú cưng (Food, có tồn kho).
3. Triển khai trọn vẹn luồng nghiệp vụ thương mại điện tử end-to-end: đăng ký → đăng nhập → duyệt sản phẩm → giỏ hàng → đặt hàng → thanh toán → giao hàng → đánh giá / hoàn trả.
4. Tích hợp Firebase Authentication (Email/Password, Google Sign-In; thiết kế sẵn kết cấu cho Facebook Login) và Cloud Firestore làm cơ sở dữ liệu real-time.
5. Tích hợp cổng thanh toán điện tử VNPay (Sandbox) sử dụng chữ ký HMAC-SHA512 đúng chuẩn tài liệu chính thức.
6. Tích hợp Chatbot tư vấn dựa trên OpenAI Chat Completions API (gpt-4o-mini), hỗ trợ gửi văn bản, ảnh và voice-to-text.
7. Áp dụng nhất quán mẫu kiến trúc MVVM kết hợp Repository Pattern; tổ chức mã nguồn rõ ràng theo từng layer; sử dụng ViewBinding và LiveData.
8. Sản phẩm cuối có giao diện thân thiện, hỗ trợ tiếng Việt đầy đủ, đáp ứng yêu cầu phi chức năng cơ bản về hiệu năng (load < 2s) và bảo mật (không lưu mật khẩu local).

3. Đối tượng và phạm vi nghiên cứu

3.1. Đối tượng nghiên cứu

- Quy trình mua bán thú cưng và thức ăn cho thú cưng tại các cửa hàng vừa và nhỏ ở Việt Nam.
- Các nghiệp vụ thương mại điện tử cơ bản: catalog sản phẩm, giỏ hàng, đặt hàng, thanh toán, vận chuyển, hoàn trả.

- Kiến trúc Model-View-ViewModel (MVVM) kết hợp Repository Pattern trên nền tảng Android.
- Hệ sinh thái Firebase (Authentication, Cloud Firestore, Storage) và cách thức xây dựng ứng dụng real-time bằng SnapshotListener.
- Cổng thanh toán VNPAY (sandbox) với cơ chế chữ ký HMAC-SHA512 và Deep Link redirect.
- Mô hình ngôn ngữ lớn (Large Language Model) thông qua OpenAI Chat Completions API và kỹ thuật Retrieval-Augmented Generation (RAG) đơn giản.

3.2. Phạm vi nghiên cứu

- Nền tảng: Android, ngôn ngữ Java; minSdk = 24 (Android 7.0), targetSdk = 36.
- Backend: Firebase (BaaS) – không xây server riêng; chỉ sử dụng Cloud Functions ở mức đề xuất tương lai.
- Thanh toán: VNPAY Sandbox (chưa đưa lên môi trường production).
- Hỗ trợ tiếng Việt; các văn bản trong app đều bằng tiếng Việt.
- Phạm vi địa lý mô phỏng: Việt Nam, với phí vận chuyển chia theo bốn vùng (TP.HCM, Miền Nam, Miền Trung, Miền Bắc).
- Đối tượng người dùng giả định: khách hàng cá nhân và quản trị viên cửa hàng – chưa hỗ trợ tài khoản B2B.

4. Phương pháp thực hiện

Để hoàn thành đề tài, nhóm áp dụng phối hợp nhiều phương pháp nghiên cứu và phát triển phần mềm:

- Phương pháp khảo sát: nhóm khảo sát một số ứng dụng/website pet shop trong và ngoài nước (Pet Mart, Petsy, Chewy, PetSmart, các bài đăng MXH) để rút ra ưu/nhược điểm và đề xuất tính năng riêng.
- Phương pháp phân tích – thiết kế: vận dụng UML (Use Case, Activity, Sequence, State, ER) để mô hình hoá hệ thống trước khi viết code.
- Phương pháp phát triển lặp tăng dần (Incremental & Iterative): chia hệ thống thành 4 module độc lập, mỗi thành viên phụ trách 1 module và tích hợp dần.
- Phương pháp lập trình theo mẫu kiến trúc MVVM + Repository Pattern, kết hợp ViewBinding và LiveData.

- Phương pháp kiểm thử: kết hợp manual test (kịch bản chi tiết) và unit test cho các util thuần Java (VNPayHelper, ShippingHelper, PromotionManager).
- Phương pháp quản lý mã nguồn: sử dụng Git/GitHub, mỗi thành viên làm việc trên nhánh riêng, merge qua Pull Request.
- Phương pháp tham khảo: đọc và áp dụng tài liệu chính thức của Android Developers, Firebase, VNPAY, OpenAI; tham khảo các bài viết kỹ thuật trên Medium, Stack Overflow.

5. Bố cục báo cáo

Báo cáo được trình bày trong bảy chương chính (ngoài phần Mở đầu, Kết luận, Tài liệu tham khảo và các phụ lục), trong đó các Chương 3, 4, 5, 6 tương ứng với bốn module phụ trách bởi bốn thành viên trong nhóm:

- Chương 1 – Tổng quan đề tài và công nghệ sử dụng: trình bày bối cảnh, khảo sát hiện trạng, mô tả bài toán, yêu cầu chức năng/phi chức năng và các công nghệ then chốt.
- Chương 2 – Phân tích và thiết kế hệ thống: Use Case, Activity Diagram, thiết kế CSDL Cloud Firestore, kiến trúc tổng thể MVVM và thiết kế UI/UX.
- Chương 3 – Xây dựng module Tài khoản và Hồ sơ người dùng (TV1).
- Chương 4 – Xây dựng module Danh mục sản phẩm và Trang chủ (TV2).
- Chương 5 – Xây dựng module Giỏ hàng, Thanh toán và Đơn hàng (TV3).
- Chương 6 – Xây dựng module Quản trị và Chatbot tư vấn (TV4).
- Chương 7 – Kiểm thử, triển khai và đánh giá: kế hoạch kiểm thử, bảng test case theo từng module, hiệu năng – bảo mật và hướng dẫn build.

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI VÀ CÔNG NGHỆ SỬ DỤNG

1.1. Khảo sát hiện trạng các ứng dụng pet shop trên thị trường

Trước khi đặc tả yêu cầu cho ứng dụng PetShop, nhóm tiến hành khảo sát một số ứng dụng và website cùng lĩnh vực, đặc biệt tập trung vào trải nghiệm người dùng cuối, tính năng đặc thù cho thú cưng và mức độ chuyên biệt của giải pháp. Bảng 1.1 dưới đây tổng hợp ưu/nhược điểm của các giải pháp tham khảo:

Bảng 2: So sánh các giải pháp Pet shop hiện có trên thị trường

Tên / Loại	Ưu điểm	Nhược điểm
Pet Mart, Petsy (Website thuần)	Sản phẩm phong phú, có hệ thống cửa hàng vật lý hỗ trợ.	Chỉ có web, chưa tối ưu cho mobile; UX chậm; thanh toán hạn chế; không có app native; thiếu chatbot tư vấn.
Chewy, PetSmart (US)	Hệ sinh thái lớn, chương trình thành viên, giao hàng định kỳ, phục vụ rất chuyên nghiệp.	Không phục vụ thị trường Việt Nam; giao diện và phương thức thanh toán không phù hợp người dùng VN.
Bài đăng MXH (Facebook, Zalo, Chợ Tốt)	Dễ đăng tin, tiếp cận nhanh, cộng đồng người chơi sẵn có.	Không có quản lý đơn hàng, không có cơ chế bảo vệ giao dịch, không có hồ sơ thú cưng đầy đủ, không hoàn trả/hoàn tiền.
Shopee, Lazada, Tiki	Hệ thống thanh toán/giao hàng hoàn thiện; nhiều người bán.	Không chuyên về thú cưng; mỗi cá thể pet không quản lý đúng theo trạng thái AVAILABLE/RESERVED/SOLD; không có tư vấn chuyên sâu.

Từ kết quả khảo sát, nhóm rút ra hai nhóm cơ hội mà ứng dụng PetShop cần khai thác: (1) cung cấp một nền tảng tập trung, chuyên biệt cho lĩnh vực thú cưng, có quản lý từng cá thể pet; và (2) tích hợp một trợ lý AI tư vấn về chăm sóc, dinh dưỡng, lựa chọn giống – điều mà các đối thủ chưa làm. Đây cũng là hai điểm khác biệt quan trọng được nhóm đặt làm trọng tâm khi đặc tả yêu cầu.

1.2. Mô tả bài toán và yêu cầu chức năng

Ứng dụng PetShop đóng vai trò là nền tảng kết nối hai nhóm người dùng chính:

- Khách hàng (Customer): có thể duyệt sản phẩm, đặt mua, thanh toán, theo dõi đơn hàng, trao đổi với chatbot, quản lý hồ sơ và số địa chỉ.
- Quản trị viên (Admin): chịu trách nhiệm cập nhật sản phẩm, danh mục, khuyến mãi, voucher; phê duyệt đơn hàng, xử lý yêu cầu hoàn trả; theo dõi doanh thu thực tế qua bảng điều khiển real-time.

1.2.1. Yêu cầu chức năng (Functional Requirements)

Bảng 1.2 và Bảng 1.3 mô tả chi tiết các yêu cầu chức năng cho hai vai trò.

Bảng 3: Yêu cầu chức năng (FR) cho khách hàng

Mã	Tên chức năng	Mô tả ngắn
FC-01	Đăng ký tài khoản	Đăng ký Email/Mật khẩu, xác thực bằng OTP gửi qua email; tự động tạo bản ghi user trên Firestore với role = CUSTOMER.
FC-02	Đăng nhập	Hỗ trợ Email/Password, Google Sign-In (đăng ký Facebook Login đã được thiết kế sẵn cho phiên bản mở rộng).
FC-03	Quên mật khẩu	Gửi email đặt lại mật khẩu thông qua Firebase Auth.
FC-04	Trang chủ	Hiển thị banner, lời chào theo thời gian, danh mục, sản phẩm nổi bật, ô tìm kiếm, badge giỏ hàng và thông báo.
FC-05	Duyệt & lọc sản phẩm	Duyệt thú cưng và thức ăn theo danh mục; tìm kiếm theo từ khóa; lọc theo loài/loại/giá.
FC-06	Chi tiết sản phẩm	Xem ảnh slideshow, mô tả, giá, tình trạng tiêm phòng (Pet) hoặc trọng lượng/loại (Food).
FC-07	Giỏ hàng	Thêm/xoá/đổi số lượng, tự động tính tiền; mỗi pet chỉ thêm được 1 lần (cá thể duy nhất).
FC-08	Voucher / Khuyến mãi	Áp dụng voucher PERCENT/FIXED/FREESHIP và promotion tự động.
FC-09	Thanh toán	Chọn địa chỉ, tính phí ship theo vùng, chọn COD hoặc VNPay.
FC-10	Lịch sử đơn hàng	Lọc theo trạng thái; xem chi tiết; huỷ đơn khi còn ở trạng thái cho phép.

FC-11	Hoàn trả / Hoàn tiền	Yêu cầu trả hàng sau khi nhận; nhập số TK ngân hàng nếu COD.
FC-12	Thông báo	Real-time badge; danh sách thông báo (đơn hàng, khuyến mãi, hệ thống).
FC-13	Chat hỗ trợ AI	Chat với GPT-4o-mini, gửi văn bản/ảnh/voice; lưu lịch sử phiên.
FC-14	Quản lý hồ sơ	Sửa thông tin cá nhân, upload avatar, quản lý số địa chỉ.

Bảng 4: Yêu cầu chức năng (FR) cho quản trị viên

Mã	Tên chức năng	Mô tả ngắn
FA-01	Dashboard	9 thẻ thống kê real-time: doanh thu, tổng đơn, người dùng, đơn theo trạng thái.
FA-02	Quản lý người dùng	Khoá/mở khoá tài khoản, đổi role.
FA-03	Quản lý danh mục	CRUD danh mục Pet và Food.
FA-04	Quản lý thú cưng	CRUD pet với upload nhiều ảnh + video.
FA-05	Quản lý thức ăn	CRUD food, cập nhật tồn kho nhanh qua dialog.
FA-06	Quản lý đơn hàng	Cập nhật trạng thái: PENDING → CONFIRMED → ... → DELIVERED.
FA-07	Duyệt hoàn trả	Approve / Reject / Refund yêu cầu trả hàng của khách.
FA-08	Quản lý voucher	CRUD voucher; auto gửi notification khi tạo/bật.
FA-09	Quản lý khuyến mãi	CRUD promotion; áp dụng % giảm giá theo sản phẩm/danh mục.

1.2.2. Yêu cầu phi chức năng (Non-Functional Requirements)

Bảng 5: Yêu cầu phi chức năng (NFR)

Tiêu chí	Mức yêu cầu
Tương thích	Android 7.0 trở lên (API 24+); thiết kế responsive với màn hình từ 4.7 inch đến 7 inch.
Hiệu năng	Trang chủ load < 2s; SnapshotListener phản hồi < 500 ms; kích thước APK < 30 MB.
Bảo mật	Sử dụng Firebase Auth (mã hoá phía Google); HMAC-SHA512 cho VNPay; không lưu mật

	khẩu local; khoá API đặt trong local.properties.
Khả năng sử dụng	UI Material Design, hỗ trợ tiếng Việt đầy đủ, các thao tác chính không quá 3 lần chạm.
Khả năng mở rộng	Dễ dàng thay shipping API (hiện hardcoded bằng ShippingHelper) bằng GHN/GHTK; dễ tích hợp FCM cho push notification.
Bảo trì	Mã nguồn chia layer rõ ràng (model/repository/viewmodel/view/utils); đặt tên nhất quán.
Khả năng kiểm thử	ViewModel tách rời View, có thể test bằng JUnit; util thuần Java có thể test offline.

1.3. Công nghệ sử dụng

1.3.1. Ngôn ngữ Java và nền tảng Android

Nhóm chọn ngôn ngữ Java cho dự án vì các lý do: (1) Java là ngôn ngữ chính thức đầu tiên của Android, có cộng đồng lớn nhất, tài liệu phong phú, dễ dàng tìm kiếm khi gặp vấn đề; (2) các thành viên trong nhóm đã có nền tảng tốt từ các môn học trước (Lập trình hướng đối tượng, Lập trình Java, Lập trình Android căn bản); (3) Java vẫn là lựa chọn ổn định cho các dự án thương mại lớn, đặc biệt với hệ sinh thái Spring/Android.

Dự án được build với Android Gradle Plugin 8.x, sử dụng Gradle Kotlin DSL (build.gradle.kts), compileSdk = 36, minSdk = 24, targetSdk = 36. Tập cấu hình version-catalog (libs.versions.toml) được sử dụng để khai báo dependency tập trung, dễ nâng cấp.

1.3.2. Kiến trúc MVVM, ViewBinding, LiveData, ViewModel

MVVM (Model – View – ViewModel) là kiến trúc được Google chính thức khuyến nghị cho Android (theo Android Architecture Components). Ưu điểm chính:

- Tách bạch logic UI khỏi logic dữ liệu: View (Activity/Fragment) chỉ hiển thị; ViewModel giữ trạng thái và xử lý logic; Model/Repository chịu trách nhiệm I/O.
- ViewModel sống theo lifecycle Activity/Fragment, tự động giữ trạng thái khi cấu hình thay đổi (xoay màn hình).
- LiveData là một observable lifecycle-aware: View đăng ký quan sát, khi data thay đổi UI tự cập nhật và tự huỷ listener khi View bị destroy → tránh memory leak.

- ViewBinding giúp truy xuất View bằng tham chiếu type-safe thay cho findViewById, giảm sai sót và NullPointerException.

Trong dự án PetShop, mỗi tính năng có một cặp Activity/Fragment + ViewModel + Repository. Các Repository đại diện cho lớp truy cập dữ liệu (Firestore, Storage, REST API). Mô hình kiến trúc tổng thể được trình bày chi tiết ở Chương 2.

1.3.3. Firebase (Authentication, Cloud Firestore, Storage)

Firebase là nền tảng Backend-as-a-Service (BaaS) của Google được nhóm sử dụng làm backend chính, gồm ba dịch vụ:

- Firebase Authentication: quản lý xác thực với nhiều phương thức (Email/Password, Google, Phone, Facebook...). Trong dự án dùng Email/Password và Google Sign-In; Facebook Login được thiết kế sẵn kết cấu, sẵn sàng kích hoạt.
- Cloud Firestore: cơ sở dữ liệu NoSQL real-time, dạng document/collection. Hỗ trợ truy vấn phức tạp, transaction, snapshot listener, security rules. Đáp ứng tốt yêu cầu real-time của module Admin Dashboard và Notification.
- Firebase Storage: lưu trữ file (ảnh, video). Sử dụng để lưu ảnh banner, ảnh sản phẩm, video giới thiệu thú cưng và avatar.

Các lý do nhóm chọn Firebase: (1) miễn phí trong giai đoạn phát triển; (2) tích hợp sẵn SDK Android, cấu hình bằng google-services.json; (3) cho phép xây dựng nhanh các ứng dụng real-time mà không cần dựng server riêng; (4) hệ sinh thái Google có hỗ trợ Cloud Functions, Cloud Messaging cho hướng phát triển tiếp theo.

1.3.4. Đăng nhập mạng xã hội: Google Sign-In, Facebook Login

Để tăng trải nghiệm đăng ký/đăng nhập cho người dùng, ứng dụng hỗ trợ đăng nhập bằng tài khoản mạng xã hội:

- Google Sign-In: sử dụng SDK Google Play Services Auth. Người dùng nhấn nút “Đăng nhập với Google”, ứng dụng mở Intent chọn tài khoản Google trên thiết bị; sau khi user xác nhận, Google trả về ID token; ID token được đẩy lên Firebase Auth để lấy phiên đăng nhập. Nếu là user mới, hệ thống tự tạo bản ghi trong Firestore với role = CUSTOMER.
- Facebook Login: được thiết kế đồng dạng (cũng dùng OAuth credential nạp vào Firebase Auth). Cấu trúc lớp đã sẵn sàng, chỉ cần cấp App ID/App Secret từ Meta for Developers và bổ sung Facebook SDK để kích hoạt. Phiên bản hiện tại nhóm

chưa kích hoạt do giới hạn thời gian xét duyệt App từ Meta; xem mục 3.7 trong Chương 3.

1.3.5. Công thanh toán VNPay

VNPay là một trong những cổng thanh toán điện tử phổ biến nhất tại Việt Nam, hỗ trợ thanh toán qua tài khoản ngân hàng nội địa, thẻ ATM, QR Code, ví VNPay-QR. Nhóm sử dụng môi trường Sandbox của VNPay để demo, gồm:

- URL thanh toán: <https://sandbox.vnpayment.vn/paymentv2/vpcpay.html>
- Mã hash secret và Terminal Code (TMN) lưu trong local.properties.
- Chữ ký HMAC-SHA512 để bảo đảm tính toàn vẹn của tham số gửi sang VNPay.
- Deep Link callback theo lược đồ `petshop://payment/vnpay-return` → `VNPayResultActivity`.

Quy trình tích hợp được cài đặt trong lớp `VNPayHelper.java` và mô tả chi tiết trong Chương 5.

1.3.6. OpenAI API (chatbot tư vấn)

OpenAI cung cấp các API mô hình ngôn ngữ lớn (LLM). Nhóm sử dụng endpoint `/v1/chat/completions` với model `gpt-4o-mini` – cho chất lượng tốt với chi phí thấp và hỗ trợ vision (gửi ảnh). Ứng dụng PetShop dùng API này cho tính năng Chatbot tư vấn:

- Người dùng nhập câu hỏi (gõ phím hoặc voice-to-text) và có thể đính kèm ảnh.
- Trước khi gọi API, `ChatViewModel` tự động tổng hợp “context” từ Firestore (danh mục, pet, food, voucher, đơn hàng của user) và prepend vào prompt – đây là kỹ thuật RAG (Retrieval-Augmented Generation) đơn giản giúp bot trả lời chính xác về sản phẩm hiện có.
- Lịch sử hội thoại được lưu vào Firestore (`users/{uid}/sessions/{sessionId}/messages`); khách chưa đăng nhập lưu cục bộ trong `SharedPreferences` với giới hạn 50 tin.

1.3.7. Thư viện hỗ trợ: Glide, OkHttp, Gson, Material...

Bên cạnh các nền tảng chính, dự án sử dụng nhiều thư viện hỗ trợ để tăng tốc phát triển và ổn định sản phẩm:

- Glide: tải và cache ảnh từ URL/Storage hiệu quả, tự động xử lý vòng đời, hỗ trợ placeholder/error.
- OkHttp: client HTTP nhỏ gọn, được dùng để gọi OpenAI API và bất kỳ REST endpoint nào trong tương lai.
- Gson: chuyển đổi JSON $\rightleftharpoons$ POJO, dùng cho parse response OpenAI và serialize/deserialize Firestore document.
- Material Components for Android: bộ component tuân theo Material Design 3 (BottomNavigationView, TextInputLayout, MaterialButton, ...).
- ViewPager2: cho banner trượt ở Trang chủ.
- RecyclerView, CardView: hiển thị danh sách và bố cục thẻ.
- CircleImageView: avatar tròn.
- JavaMail (javax.mail): gửi email OTP qua SMTP Gmail trong RegisterActivity.
- Lifecycle ViewModel + LiveData (androidx.lifecycle:*) : cốt lõi của MVVM.

1.4. Công cụ phát triển và quản lý dự án

1.4.1. Android Studio, Gradle (Kotlin DSL)

Toàn bộ quá trình phát triển được thực hiện trên Android Studio (phiên bản Hedgehog 2023.x trở lên), kết hợp Gradle 8 với cú pháp Kotlin DSL (build.gradle.kts) – có lợi thế hơn Groovy về kiểm tra cú pháp và auto-complete. Tập libs.versions.toml dùng để khai báo dependency tập trung, đảm bảo tất cả module dùng chung phiên bản.

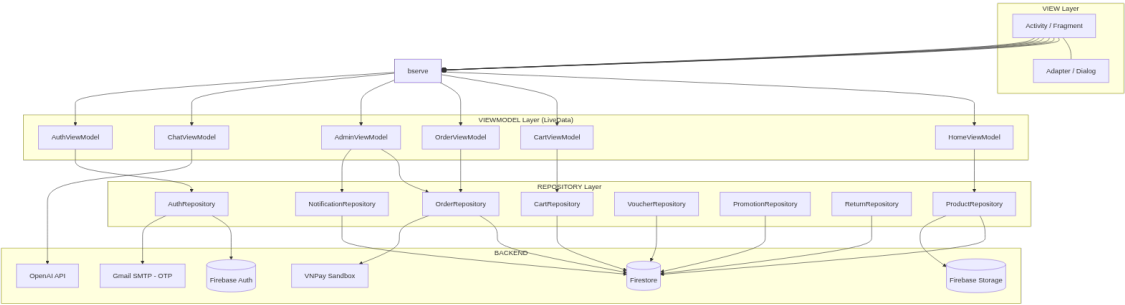
1.4.2. Git, GitHub, quản lý nhánh

Mã nguồn được quản lý bằng Git và lưu trên GitHub. Nhóm sử dụng quy trình Git-flow rút gọn: nhánh chính main giữ trạng thái ổn định; mỗi tính năng phát triển trên một nhánh feature/<tên-tính-năng>; sau khi hoàn thành sẽ mở Pull Request để review trước khi merge. Quy ước commit ngắn gọn theo dạng “fix:” / “feat:” / “docs:” giúp truy vết lịch sử dễ dàng.

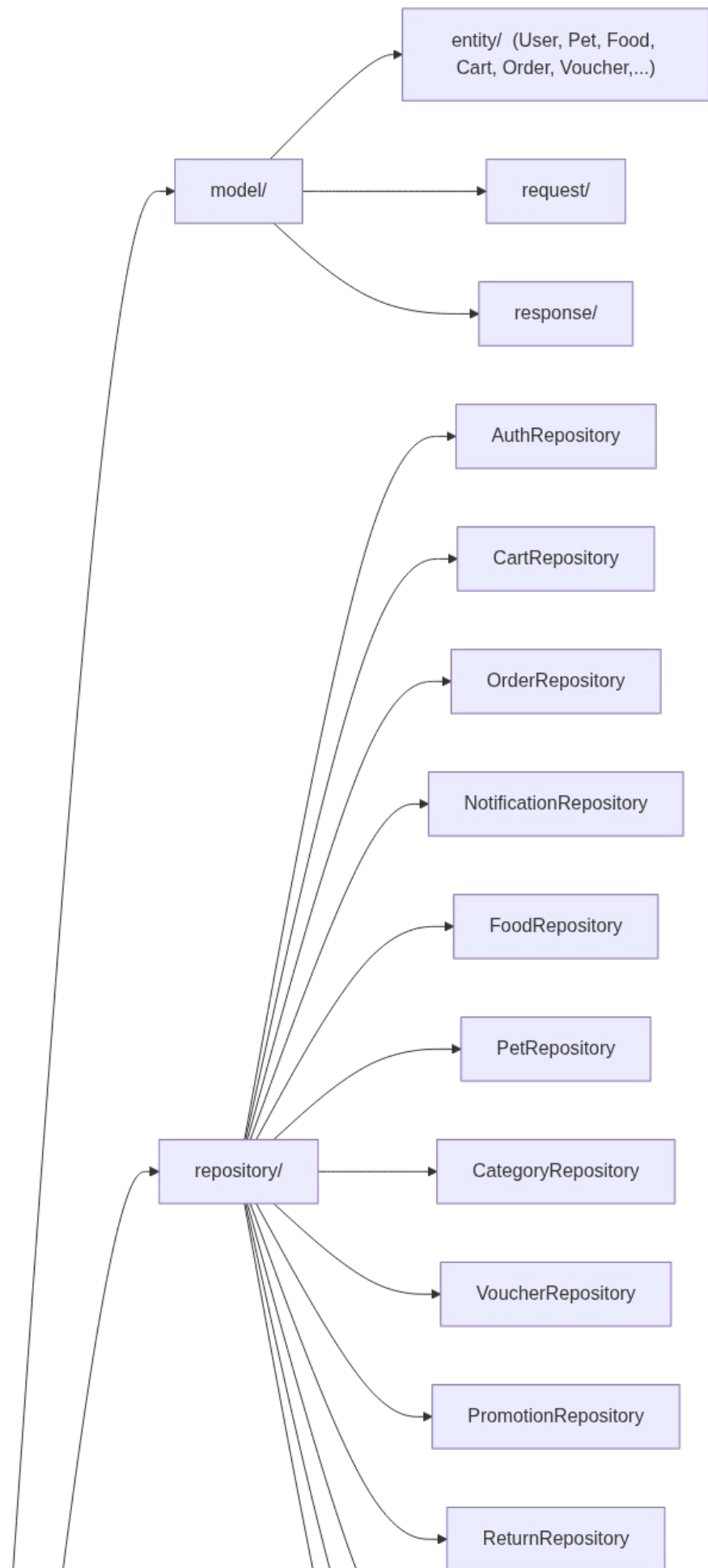
1.4.3. Figma cho thiết kế giao diện

Nhóm sử dụng Figma để vẽ wireframe và mockup các màn hình trước khi viết XML layout. Việc thống nhất bộ màu (cam #F5A623, nâu nhạt #FFF8F2, trắng), font, icon trên Figma giúp các thành viên không cần trao đổi nhiều mà vẫn cho ra giao diện

đồng nhất. Một số màn hình tiêu biểu (Splash, Login, Home, Cart, Checkout) được prototype trên Figma để demo cho giảng viên trước khi triển khai chính thức.



Hình 1: Sơ đồ kiến trúc MVVM + Repository + Firebase của ứng dụng PetShop

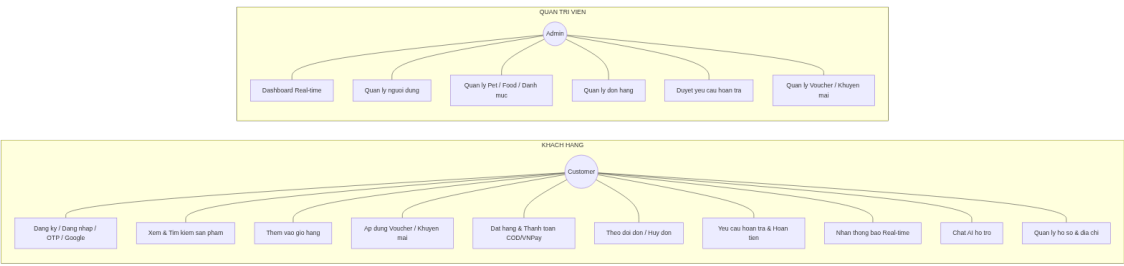


Hình 2: Mô hình tổ chức package mã nguồn theo MVVM

CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

2.1. Sơ đồ Use Case tổng quát

Sơ đồ Use Case tổng quát của ứng dụng PetShop được trình bày trong Hình 1.1 (đã đặt tại Chương 1). Sơ đồ thể hiện rõ hai tác nhân của hệ thống và các Use Case mà mỗi tác nhân tương tác.



Hình 3: Sơ đồ Use Case tổng quát của ứng dụng PetShop

2.1.1. Tác nhân Khách hàng

Khách hàng là tác nhân chính, đại diện cho người dùng cuối có nhu cầu mua thú cưng và thức ăn. Một khách hàng có thể đăng ký tài khoản hoặc duyệt sản phẩm ở trạng thái guest (chưa đăng nhập). Tuy nhiên, các thao tác liên quan đến giỏ hàng, đơn hàng, thông báo cá nhân, chat lưu trữ phiên... đều yêu cầu đăng nhập. Khách hàng có thể giữ nhiều địa chỉ giao hàng khác nhau và chọn địa chỉ mặc định.

2.1.2. Tác nhân Quản trị viên

Quản trị viên là người được cấp role = ADMIN trên Firestore. Sau khi đăng nhập, hệ thống điều hướng admin tới giao diện AdminActivity (khác hoàn toàn với giao diện của khách hàng). Admin chịu trách nhiệm vận hành cửa hàng: cập nhật catalog, duyệt đơn, giải quyết hoàn trả, theo dõi doanh thu. Hệ thống không cho phép admin tự đặt hàng cho mình – tách biệt rõ vai trò.

2.2. Đặc tả các Use Case chính

Để minh họa chi tiết, nhóm trình bày đặc tả của hai Use Case quan trọng nhất: Đặt hàng & Thanh toán (Customer) và Yêu cầu trả hàng / hoàn tiền (Customer–Admin).

Bảng 6: Mô tả Use Case 'Đặt hàng và thanh toán'

Trường	Nội dung
Tên Use Case	UC-05: Đặt hàng và thanh toán

Mô tả	Khách hàng tạo đơn hàng từ giỏ hàng, chọn địa chỉ, phương thức thanh toán và xác nhận thanh toán.
Tác nhân	Customer
Tiền điều kiện	Đã đăng nhập; giỏ hàng có ít nhất 1 sản phẩm; có ít nhất 1 địa chỉ.
Hậu điều kiện	Đơn hàng được tạo ở trạng thái PENDING (COD) hoặc WAITING_PAYMENT (VNPay); tồn kho food được trừ; pet chuyển sang RESERVED khi PAID.
Luồng chính	1) Mở giỏ hàng → nhấn Thanh toán. 2) CheckoutActivity hiển thị danh sách sản phẩm, địa chỉ, phí ship. 3) Khách chọn voucher (tùy chọn) và phương thức thanh toán. 4) Nếu COD → OrderRepo.createOrder(); chuyển sang OrderDetail. 5) Nếu VNPAY → tạo URL bằng VNPayHelper, mở VNPayWebView; sau khi thanh toán → completeVNPayOrder() chốt đơn.
Luồng phụ	A1: Voucher quá hạn / vượt giới hạn → báo lỗi, không áp dụng. A2: VNPAY thất bại → đơn vẫn ở WAITING_PAYMENT, khách có thể thanh toán lại. A3: Mất kết nối → giao dịch tạo đơn rollback nhờ Firestore Transaction.
Ngoại lệ	E1: Pet đã bị reserved/sold trong lúc thanh toán → transaction fail, báo lỗi tới khách. E2: Stock food không đủ → báo lỗi, đề nghị giảm số lượng.

Bảng 7: Mô tả Use Case 'Yêu cầu trả hàng / hoàn tiền'

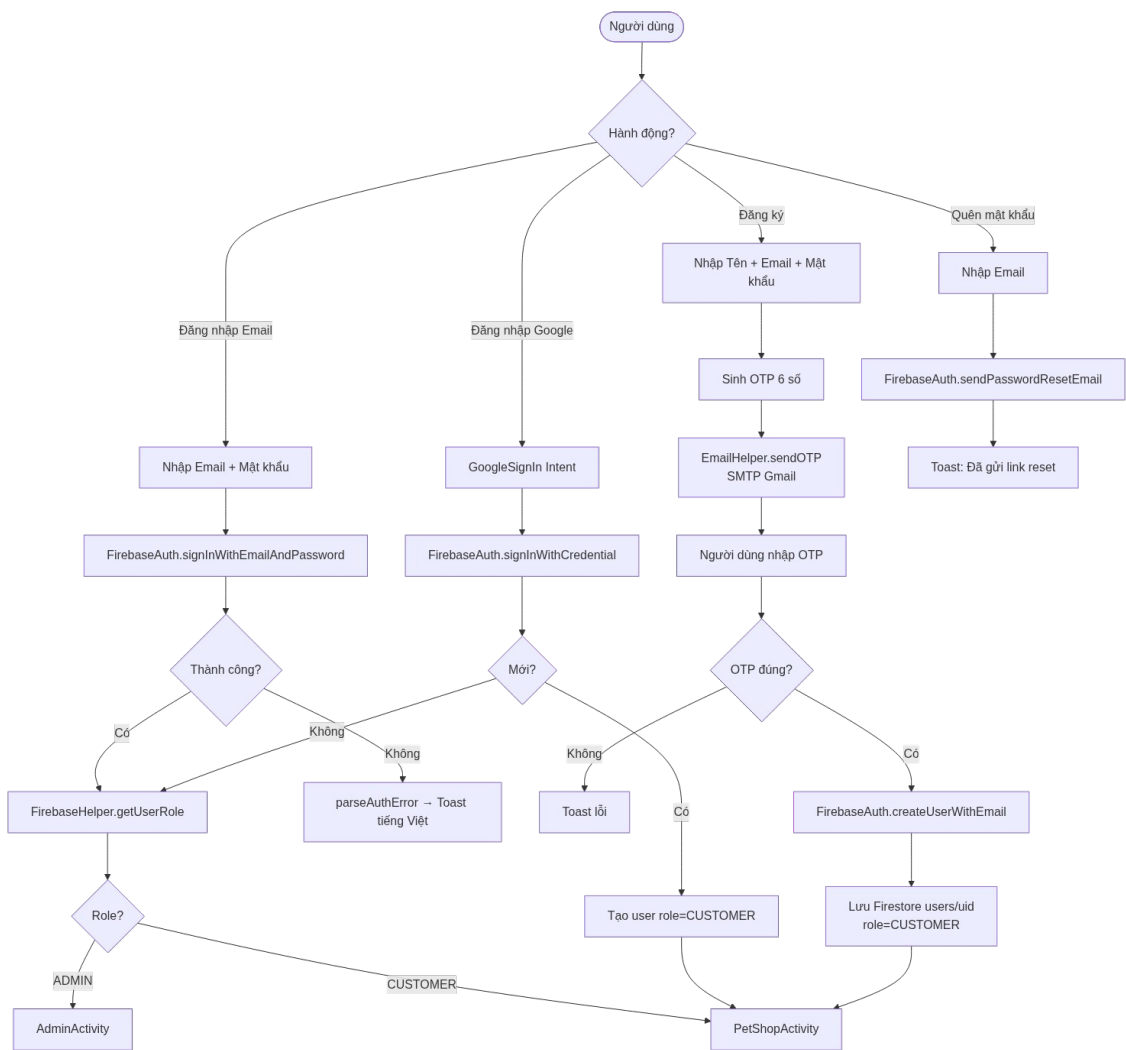
Trường	Nội dung
Tên Use Case	UC-09: Yêu cầu trả hàng và hoàn tiền
Mô tả	Khách hàng yêu cầu trả hàng/hoàn tiền sau khi nhận hàng; admin duyệt và xử lý hoàn tiền.
Tác nhân	Customer (gửi yêu cầu); Admin (xử lý)
Tiền điều kiện	Đơn hàng đã ở trạng thái DELIVERED hoặc COMPLETED.
Hậu điều kiện	Đơn chuyển sang RETURN_REQUESTED → RETURN_APPROVED → REFUNDED; doanh thu hệ thống tự giảm tương ứng.
Luồng chính	1) Customer mở chi tiết đơn → nhấn 'Yêu cầu hoàn trả'. 2) Nhập lý do; nếu COD bắt buộc nhập số tài

	khoản + tên ngân hàng. 3) OrderRepo.requestReturn() cập nhật trạng thái; ReturnRepo.create() lưu yêu cầu. 4) Notification gửi đến admin (qua tile real-time). 5) Admin mở AdminReturnList → Approve / Reject. 6) Nếu Approve → admin chuyển khoản (COD) hoặc gọi API refund VNPay (tương lai), sau đó cập nhật REFUNDED.
Luồng phụ	A1: Admin từ chối → đơn quay lại COMPLETED + ghi adminNote.
Ngoại lệ	E1: Yêu cầu trả hàng quá thời hạn quy định → ấn nút.

2.3. Sơ đồ hoạt động (Activity Diagram) các luồng nghiệp vụ tiêu biểu

2.3.1. Luồng Đăng ký – Đăng nhập

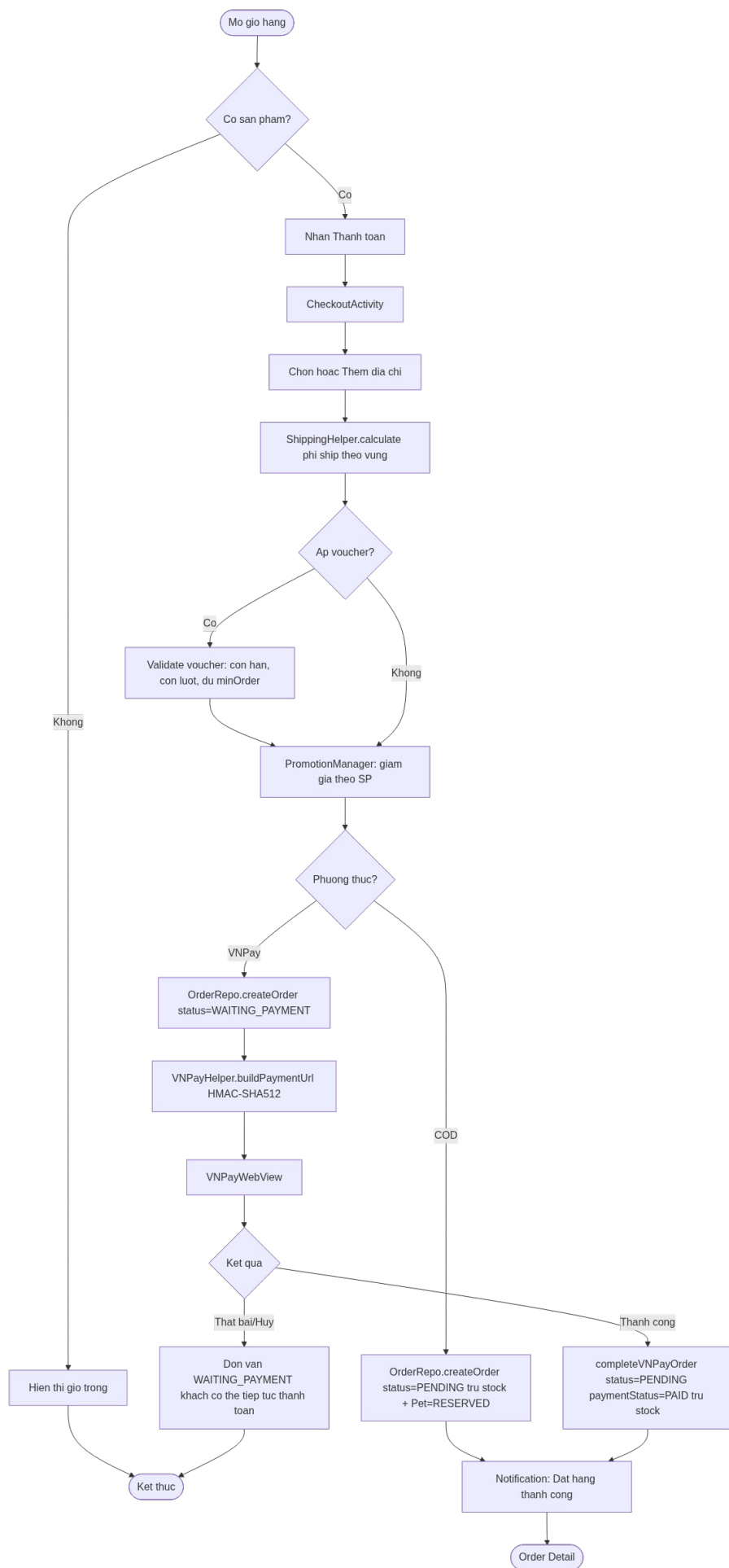
Luồng đăng ký – đăng nhập của ứng dụng được mô tả trong Hình kèm theo. Hệ thống hỗ trợ ba con đường vào: (a) đăng nhập email/mật khẩu; (b) đăng nhập Google; (c) đăng ký mới có xác thực OTP qua email. Sau khi xác thực thành công, hệ thống đọc role từ Firestore để điều hướng tới giao diện Admin hoặc Customer.



Hình 4: Activity Diagram – luồng đăng ký, đăng nhập

2.3.2. Luồng Mua hàng – Thanh toán VNPay

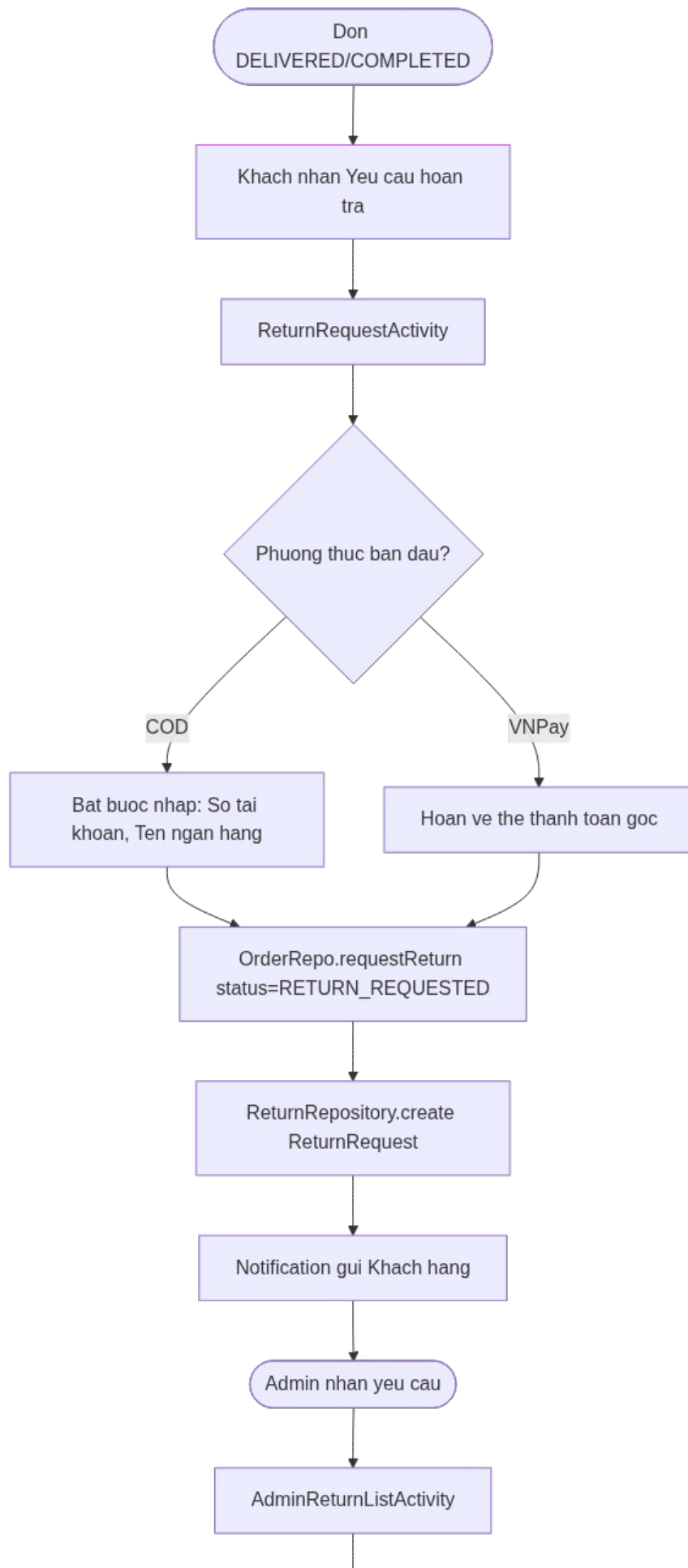
Luồng mua hàng được tổ chức theo các bước: chọn địa chỉ → tính phí ship → áp voucher/promotion → chọn phương thức thanh toán. Riêng luồng VNPay phức tạp hơn vì phải tạo URL có chữ ký HMAC-SHA512, mở WebView, đợi VNPay redirect về deep link và chốt đơn bằng transaction. Chi tiết được mô tả trong Hình kèm theo.



Hình 5: Activity Diagram – luồng đặt hàng & thanh toán

2.3.3. Luồng Yêu cầu trả hàng

Luồng yêu cầu trả hàng có hai giai đoạn: (1) phía khách hàng tạo yêu cầu, hệ thống cập nhật trạng thái đơn sang RETURN_REQUESTED; (2) phía admin xử lý: Approve → Refund hoặc Reject. Khi Refund, hệ thống tự động trừ doanh thu khỏi dashboard và gửi notification cho khách.

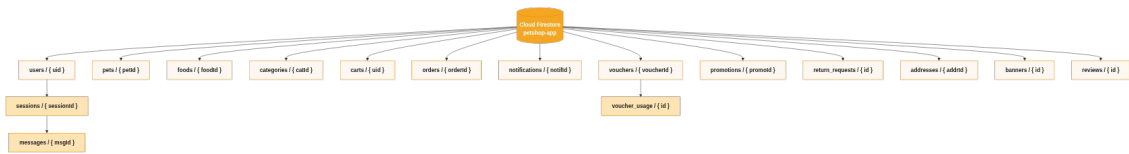


Hình 6: Activity Diagram – yêu cầu trả hàng và hoàn tiền

2.4. Thiết kế cơ sở dữ liệu trên Cloud Firestore

2.4.1. Sơ đồ collection / document

Khác với các CSDL quan hệ truyền thống, Cloud Firestore là một CSDL NoSQL tổ chức theo mô hình collection/document. Mỗi collection chứa các document, mỗi document có thể chứa các sub-collection. Sơ đồ dưới đây thể hiện toàn bộ collection của ứng dụng PetShop.



Hình 7: Sơ đồ collection Firestore của ứng dụng PetShop

Một số collection có sub-collection:

- users/{uid}/sessions/{sessionId}/messages – lưu lịch sử chat từng phiên hội thoại.
- vouchers/{voucherId}/voucher_usage – lưu lịch sử lượt sử dụng voucher của từng user.

2.4.2. Mô tả các collection

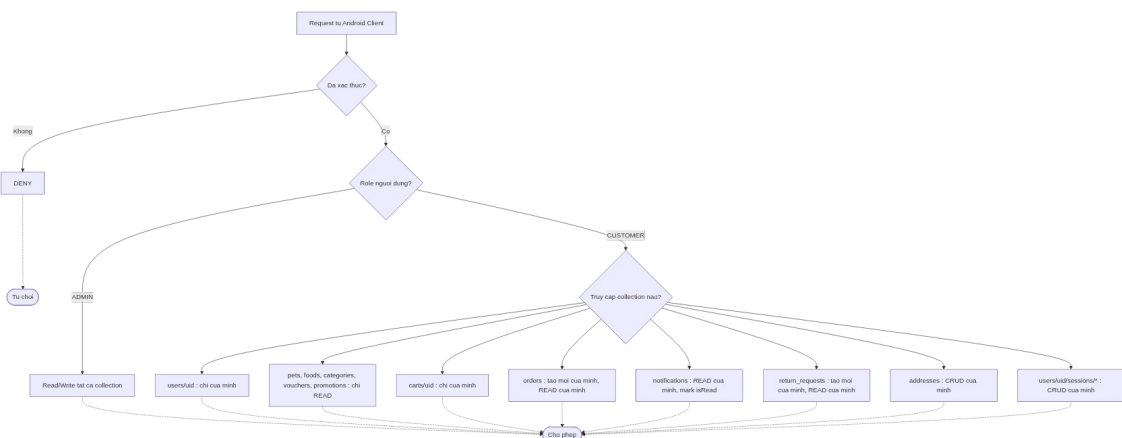
Bảng 8: Cấu trúc các collection chính trên Firestore

Collection	Mô tả ngắn	Trường tiêu biểu
users	Tài khoản người dùng (CUSTOMER/ADMIN).	id, fullName, email, role, status, totalOrders, totalSpent, avatarUrl
pets	Thú cưng đang bán (mỗi pet là một cá thể).	id, name, species, breed, age, gender, price, status (AVAILABLE/RESERVED/OLD), media[]
foods	Sản phẩm thức ăn cho thú cưng.	id, name, brand, foodType, targetPetType, weightGram, price, stock, sold
categories	Danh mục Pet/Food.	id, name, type (PET/FOOD), iconUrl, isActive
carts	Giỏ hàng (mỗi user có document cùng id = uid).	userId, items[], subtotal, totalItems, updatedAt
orders	Đơn hàng đã đặt.	id, orderCode, userId, items[], shippingAddress, totalAmount, status, paymentMethod, paymentStatus
notifications	Thông báo gửi cho user.	id, userId, type

		(ORDER/PROMO/SYSTEM), title, message, isRead, createdAt, orderId
vouchers	Mã giảm giá hệ thống.	id, code, type (PERCENT/FIXED/FREESHIP), discountValue, usageLimit, usedCount
promotions	Khuyến mãi áp dụng tự động cho sản phẩm.	id, name, applyType (ALL/CATEGORY/PRODUCT), discountValue, startDate, endDate
return_requests	Yêu cầu trả hàng & hoàn tiền.	id, orderId, userId, reason, paymentMethod, bankAccount, status, refundAmount
addresses	Số địa chỉ giao hàng.	id, userId, receiverName, phone, address, ward, district, city, isDefault
banners	Banner trang chủ.	id, imageUrl, link, sortOrder, isActive
reviews	Đánh giá sản phẩm sau khi mua.	id, productId, userId, rating, comment, media[], createdAt

2.4.3. Quy tắc bảo mật (Security Rules)

Việc đặt Cloud Firestore Security Rules đúng và chặt chẽ là yếu tố then chốt để bảo vệ dữ liệu khi client truy cập trực tiếp Firestore. Nhóm thiết kế Security Rules theo nguyên tắc “mặc định từ chối, chỉ cho phép có chủ đích”, được khái quát trong Hình kèm theo.



Hình 8: Tóm tắt logic Firestore Security Rules

Một số luật chính (rút gọn, cú pháp Firestore Rules):

```
rules_version = '2';
service cloud.firestore {
```

```

match /databases/{database}/documents {

  function isSignedIn() { return request.auth != null; }
  function isOwner(uid) { return isSignedIn() && request.auth.uid == uid; }
  function isAdmin() {
    return isSignedIn()
      && get(/databases/{database}/documents/users/{
(request.auth.uid)})
      .data.role == 'ADMIN';
  }

  // users/{uid}: customer chi sua duoc cua minh; admin co toan quyen
  match /users/{uid} {
    allow read, write: if isOwner(uid) || isAdmin();
    match /sessions/{sid}/messages/{mid} {
      allow read, write: if isOwner(uid);
    }
  }

  // pets, foods, categories, vouchers, promotions: customer chi READ
  match /{col=pets|foods|categories|vouchers|promotions|banners}/{id}
{
  allow read: if isSignedIn();
  allow write: if isAdmin();
}

  // carts/{uid}: chi cua minh
  match /carts/{uid} {
    allow read, write: if isOwner(uid);
  }

  // orders: customer tao moi don cua minh, READ cua minh; admin toan quyen
  match /orders/{oid} {
    allow read: if isAdmin() || (isSignedIn() &&
resource.data.userId == request.auth.uid);
    allow create: if isSignedIn() && request.resource.data.userId ==
request.auth.uid;
    allow update: if isAdmin();
  }

  // notifications: chi nguoi nhan READ va mark isRead; admin write
  match /notifications/{nid} {
    allow read: if isAdmin() || (isSignedIn() &&
resource.data.userId == request.auth.uid);
    allow create: if isAdmin();
    allow update: if isAdmin() || (isSignedIn() &&
resource.data.userId == request.auth.uid);
  }

  // return_requests: customer tao cua minh; admin xu ly
  match /return_requests/{rid} {
    allow read: if isAdmin() || (isSignedIn() &&

```

```

resource.data.userId == request.auth.uid);
    allow create: if isSignedIn() && request.resource.data.userId ==
request.auth.uid;
    allow update: if isAdmin();
  }

  // addresses: chỉ của mình
  match /addresses/{aid} {
    allow read, write: if isSignedIn() && resource.data.userId ==
request.auth.uid;
    allow create:      if isSignedIn() && request.resource.data.userId
== request.auth.uid;
  }
}
}

```

Bộ rules trên đảm bảo: (1) khách hàng tuyệt đối không truy cập được dữ liệu của người khác; (2) khách không thể tự cập nhật trạng thái đơn (chỉ admin); (3) admin có toàn quyền nhưng phải đăng nhập; (4) các collection danh mục có thể đọc tự do (đã đăng nhập) nhưng chỉ admin được sửa. Bộ rules đầy đủ được lưu trong Phụ lục C.

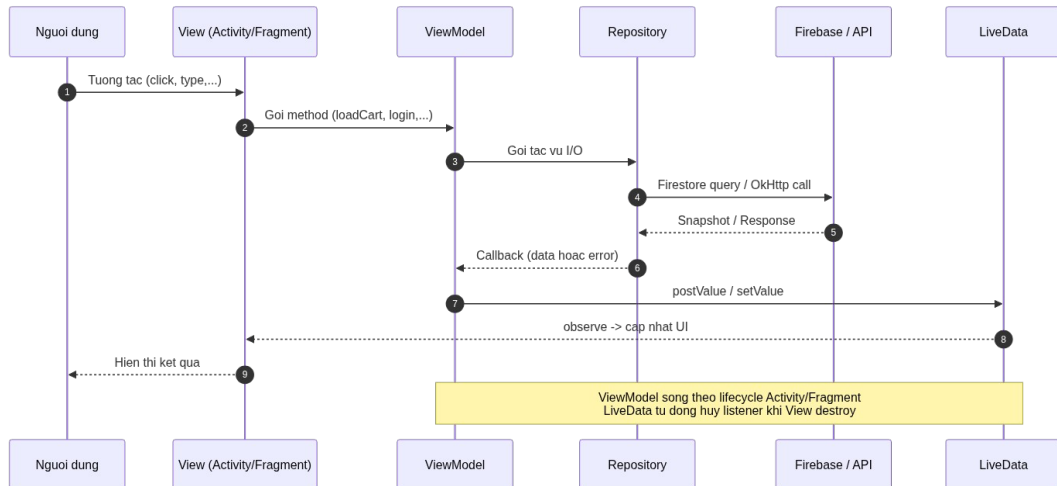
2.5. Kiến trúc tổng thể của ứng dụng

2.5.1. Mô hình MVVM trong dự án

Như đã trình bày ở Chương 1, dự án áp dụng nhất quán mô hình MVVM kết hợp Repository Pattern. Cụ thể trong dự án PetShop, vai trò các tầng như sau:

- Model (model/entity, model/request, model/response): các POJO ánh xạ document Firestore và DTO nội bộ. Ví dụ: User, Pet, Food, Cart, Order, Voucher, Notification...
- Repository (repository/): chịu trách nhiệm I/O với Firestore (CRUD, transaction, snapshot listener), Storage (upload ảnh) và REST API (OkHttp). Mỗi repository chỉ phụ thuộc Firestore SDK, không phụ thuộc Android UI.
- ViewModel (viewmodel/): giữ trạng thái màn hình bằng MutableLiveData; gọi Repository, transform/aggregate dữ liệu rồi postValue. ViewModel KHÔNG biết tới Activity/Fragment cụ thể.
- View (view/activity, view/fragment, view/adapter, view/dialog): hiển thị UI và observe LiveData; chỉ gọi method của ViewModel khi người dùng tương tác.
- Utils (utils/): các lớp hỗ trợ chung như VNPayHelper (build URL HMAC-SHA512), EmailHelper (gửi OTP), ShippingHelper (tính phí ship), PromotionManager, StorageHelper...

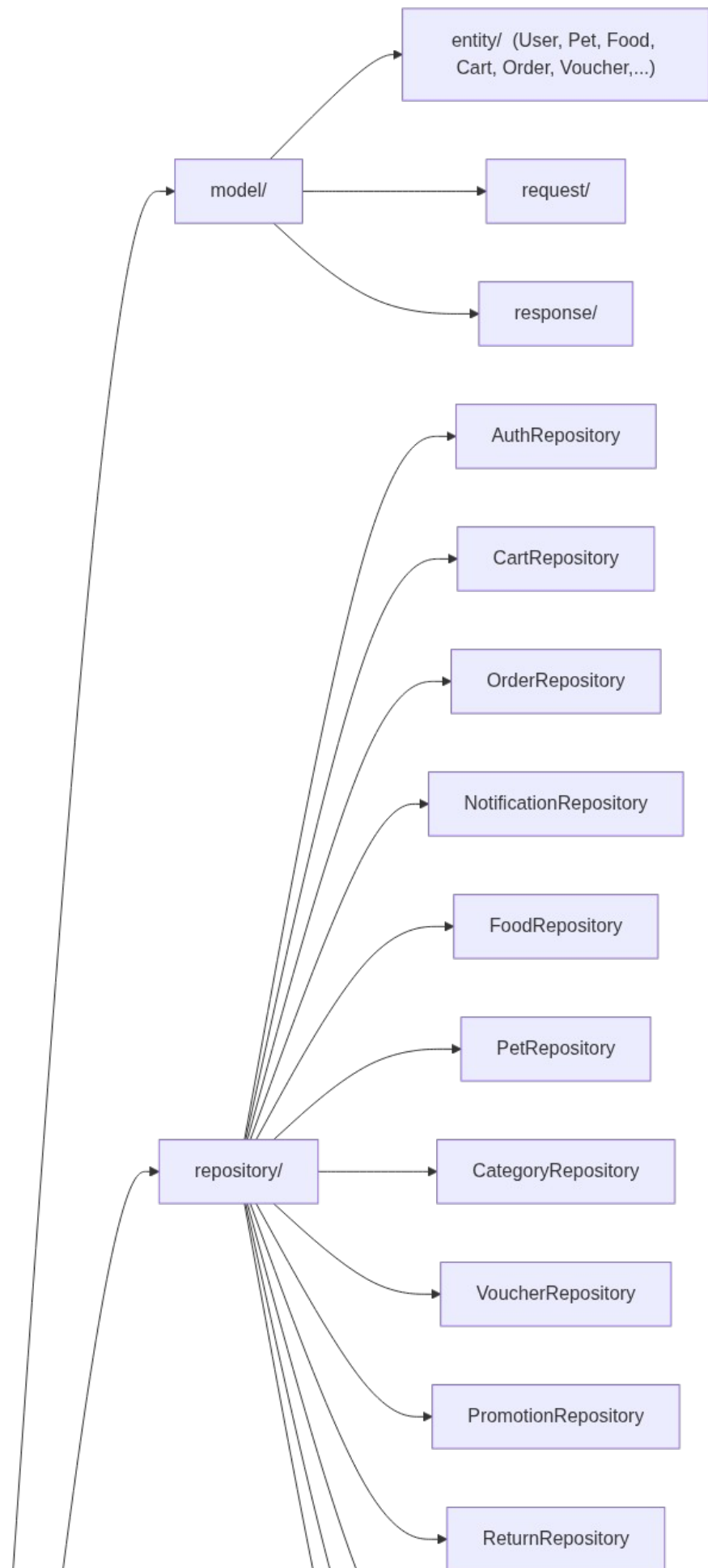
2.5.2. Sơ đồ luồng dữ liệu giữa các tầng



Hình 9: Sơ đồ luồng dữ liệu giữa các tầng trong MVVM

Sơ đồ thể hiện chu trình dữ liệu chuẩn: View → ViewModel → Repository → Firebase/API → callback ngược lại → ViewModel → LiveData → View. Cả chu trình diễn ra bất đồng bộ; LiveData đảm bảo việc cập nhật UI luôn xảy ra trên main thread và an toàn lifecycle.

2.5.3. Tổ chức package và quy ước đặt tên



Hình 10: Tổ chức package mã nguồn theo MVVM và Repository

Các quy ước đặt tên trong dự án:

- Activity: <Tên>Activity.java (vd: LoginActivity, CheckoutActivity).
- Fragment: <Tên>Fragment.java.
- ViewModel: <Tên>ViewModel.java – mỗi tính năng/module một ViewModel.
- Repository: <Tên>Repository.java – tách theo domain.
- Adapter: <Tên>Adapter.java – kèm interface onItemClickListener nếu cần.
- Layout XML: activity_<ten>.xml / fragment_<ten>.xml / item_<ten>.xml / dialog_<ten>.xml.
- Resource id: theo dạng tvName, etEmail, btnLogin, ivAvatar, rvList...
- Hằng số: viết HOA_GẠCH_DƯỚI, đặt trong Constants.java hoặc trong chính lớp sở hữu.

2.6. Thiết kế giao diện (UI/UX)

2.6.1. Bộ màu, font, icon, theme

Giao diện được thiết kế theo phong cách Material Design 3 với bảng màu lấy cảm hứng từ tông cam ấm – gợi cảm giác thân thiện, gần gũi với thú cưng (file colors.xml):

- Primary: #F5A623 (cam đậm), Primary Dark: #E09010, Primary Light: #FFD580.
- Background chính: #FFF8F2 (kem nhạt), Card: #FFFFFF.
- Text Primary: #1A1A1A, Text Secondary: #888888, Hint: #AEAEB2.
- Status: Success #34C759, Error #FF3B30, Warning #FF9500.
- Font: Roboto (hệ thống) cho UI, Times New Roman cho báo cáo.
- Icon: Material Icons + một số icon vẽ riêng (ic_paw, ic_cart, ic_chat,...).

2.6.2. Wireframe / Mockup các màn hình chính

Trước khi viết XML layout, nhóm vẽ wireframe các màn hình chính trên Figma, bao gồm: Splash, Login, Register, Home, Pet/Food Detail, Cart, Checkout, Order, Profile (cho Customer); Dashboard, Manage Users/Pets/Foods/Vouchers (cho Admin). Wireframe được giảng viên duyệt trước khi nhóm bắt tay vào code, giúp giảm thiểu việc phải sửa giao diện sau này. Một số ảnh chụp màn hình thực tế của ứng dụng được trình bày trong Phụ lục D.

[CHỤP ẢNH 2.6] Đề nghị nhóm chèn 1–2 mockup tiêu biểu của Figma tại đây để minh hoạ giai đoạn thiết kế.

CHƯƠNG 3. XÂY DỰNG MODULE TÀI KHOẢN VÀ HỒ SƠ NGƯỜI DÙNG

Thành viên phụ trách: TV1 – Nguyễn Hữu Đức Thọ (Trưởng nhóm).

3.1. Phân tích chức năng

Module Tài khoản và Hồ sơ người dùng là “cánh cửa” đầu tiên của ứng dụng – mọi tính năng riêng tư của người dùng (giỏ hàng, đơn hàng, thông báo, chat lưu trữ) đều phụ thuộc vào module này. Vì vậy module được đặt trong tay TV1 với yêu cầu cao về độ ổn định và bảo mật.

Module gồm các tính năng:

- Splash Screen và điều hướng khởi động.
- Đăng ký tài khoản bằng Email/Mật khẩu, có xác thực OTP qua email.
- Đăng nhập bằng Email/Mật khẩu.
- Đăng nhập bằng tài khoản Google.
- Đăng nhập bằng tài khoản Facebook (thiết kế sẵn cho phiên bản mở rộng).
- Quên mật khẩu (gửi link reset qua Firebase Auth).
- Quản lý hồ sơ cá nhân: avatar, họ tên, số điện thoại, ngày sinh, giới tính.
- Quản lý số địa chỉ giao hàng (CRUD, đặt mặc định).
- Hệ thống thông báo (Notification): real-time badge, danh sách, mark as read.
- Quản lý phiên đăng nhập bằng SessionManager + SharedPreferences.
- Đăng xuất, xoá phiên cục bộ.

3.2. Thiết kế lớp Model: User, Address, Notification

Để biểu diễn dữ liệu người dùng, nhóm thiết kế ba lớp Model chính. Bảng 3.1 mô tả chi tiết các thuộc tính của Model User – cũng là document trên collection users của Firestore.

Bảng 9: Các thuộc tính của Model User

Trường	Kiểu	Mô tả
id	String	UID do Firebase Auth sinh ra.
fullName	String	Họ và tên đầy đủ.
email	String	Email đăng nhập / liên hệ.
phone	String	Số điện thoại.

avatarUrl	String	URL ảnh đại diện trên Firebase Storage.
role	String	ADMIN CUSTOMER.
loginType	String	EMAIL GOOGLE FACEBOOK.
status	String	ACTIVE INACTIVE BANNED.
gender	String	MALE FEMALE OTHER.
dateOfBirth	String	yyyy-MM-dd.
totalOrders	int	Số đơn đã hoàn thành.
totalSpent	double	Tổng số tiền đã chi tiêu.
createdAt	String	Thời điểm tạo tài khoản.
updatedAt	String	Thời điểm cập nhật gần nhất.

Lớp Address mô tả một địa chỉ giao hàng, bao gồm: id, userId, receiverName, phone, address (số nhà + đường), ward, district, city, isDefault. Lớp Notification gồm: id, userId, title, message, type (ORDER/PROMO/SYSTEM), isRead, createdAt, orderId. Notification được thiết kế đơn giản nhưng đủ thông tin để client điều hướng (ví dụ click vào notification kiểu ORDER sẽ mở chi tiết đơn hàng tương ứng).

3.3. Cấu hình Firebase Authentication

Để sử dụng Firebase Authentication, nhóm thực hiện các bước cấu hình sau:

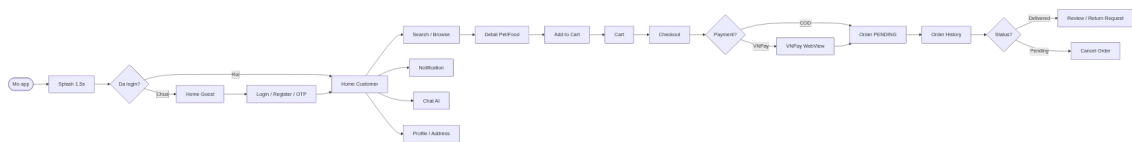
9. Tạo project trên Firebase Console (<https://console.firebase.google.com>), thêm app Android với package name com.example.petshop.
10. Tải file google-services.json về thư mục app/ của dự án.
11. Trong Authentication > Sign-in method bật Email/Password và Google. Riêng Google cần khai báo SHA-1 fingerprint của keystore debug và keystore release.
12. Thêm dependency Firebase BoM, Auth, Firestore, Storage và Google Play Services Auth vào libs.versions.toml.
13. Trong build.gradle.kts của module app, áp dụng plugin com.google.gms.google-services.
14. Khai báo OAuth client ID Web trong local.properties (GOOGLE_WEB_CLIENT_ID) và truyền vào BuildConfig để dùng khi tạo GoogleSignInOptions.

3.4. Màn hình Splash và điều hướng khởi động

SplashActivity là entry-point đầu tiên (khai báo intent-filter MAIN/LAUNCHER trong AndroidManifest.xml). Màn hình hiển thị logo và slogan “Find Your Furry Favorite” trong ~1.5 giây. Trong khoảng thời gian này, ứng dụng kiểm tra:

- Nếu SessionManager.isLoggedIn() == false → mở PetShopActivity (giao diện guest).
- Nếu đã đăng nhập và role == ADMIN → mở AdminActivity.
- Nếu đã đăng nhập và role == CUSTOMER → mở PetShopActivity.

Cách điều hướng này tách biệt rõ ràng hai role; phía customer không thể truy cập màn hình admin và ngược lại.



Hình 11: Hành trình người dùng (Customer Journey)

3.5. Đăng ký – Đăng nhập bằng Email/Mật khẩu

Quy trình đăng ký tài khoản gồm hai bước, trong đó bước OTP qua email được nhóm tự cài đặt thay vì dùng SMS phí cao:

15. Người dùng nhập họ tên, email, mật khẩu và xác nhận mật khẩu.
16. Nhấn “Gửi OTP” → ứng dụng gọi `FirebaseAuth.fetchSignInMethodsForEmail()` để kiểm tra email đã tồn tại chưa. Nếu chưa, sinh OTP 6 số ngẫu nhiên, lưu tạm trong biến và gọi `EmailHelper.sendOTP()` (sử dụng `JavaMail SMTP Gmail`) để gửi.
17. Người dùng nhập OTP nhận được, nhấn “Đăng ký”. Hệ thống so sánh OTP; nếu trùng khớp → gọi `FirebaseAuth.createUserWithEmailAndPassword()` và lưu document user vào `Firestore`.
18. Nếu đăng ký thành công → mở `PetShopActivity` với role `CUSTOMER`.

Đoạn mã rút gọn của `EmailHelper.sendOTP()`:

```
public static void sendOTP(String toEmail, String otp, EmailCallback cb)
{
    final String senderEmail    = "yourshop@gmail.com";
    final String senderPassword = "<APP_PASSWORD_GMAIL>";

    Properties props = new Properties();
    props.put("mail.smtp.auth", "true");
    props.put("mail.smtp.starttls.enable", "true");
```



```

        props.put("mail.smtp.host", "smtp.gmail.com");
        props.put("mail.smtp.port", "587");

        Session session = Session.getInstance(props, new Authenticator() {
            @Override protected PasswordAuthentication
getPasswordAuthentication() {
                return new PasswordAuthentication(senderEmail,
senderPassword);
            }
        });

        new Thread(() -> {
            try {
                Message message = new MimeMessage(session);
                message.setFrom(new InternetAddress(senderEmail));
                message.setRecipients(Message.RecipientType.TO,
InternetAddress.parse(toEmail));
                message.setSubject("Ma xac thuc dang ky Petshop");
                message.setText("Ma OTP cua ban la: " + otp + ". Khong chia
se cho ai.");
                Transport.send(message);
                cb.onSuccess();
            } catch (MessagingException e) {
                cb.onFailure("Loi gui email: " + e.getMessage());
            }
        }).start();
    }
}

```

Việc gửi mail được thực hiện trên thread riêng để không chặn UI thread. App password Gmail được cấp riêng cho ứng dụng gửi OTP và có thể vô hiệu hoá bất cứ lúc nào.

Phía đăng nhập email/mật khẩu được xử lý trong AuthViewModel.loginWithEmail(), gọi FirebaseHelper.loginWithEmail() để kết nối tới FirebaseAuth, sau đó truy vấn role để điều hướng đúng màn hình.

[CHỤP ẢNH 3.5] Đề nghị chèn ảnh các bước: (a) màn hình Register trống; (b) sau khi nhập OTP; (c) màn hình Login email; (d) thông báo lỗi sai mật khẩu.

3.6. Đăng nhập bằng Google

Đăng nhập Google sử dụng SDK Google Play Services Auth. Quy trình:

19. Khởi tạo GoogleSignInClient với GoogleSignInOptions.Builder().requestIdToken(GOOGLE_WEB_CLIENT_ID).requestEmail().build().

20. Khi user nhấn nút Google, gọi `googleSignInClient.signInIntent()` và mở qua `ActivityResultLauncher`.
21. Nhận callback, lấy `GoogleSignInAccount` và `idToken`.
22. Truyền `idToken` vào `AuthViewModel.loginWithGoogle()` →
`FirebaseHelper.loginWithGoogle()` → tạo
`GoogleAuthProvider.getCredential(idToken, null)` →
`FirebaseAuth.signInWithCredential()`.
23. Nếu là user mới → lưu vào Firestore với role CUSTOMER. Nếu là user cũ → lấy role hiện tại để điều hướng.

[CHỤP ẢNH 3.6] Đề nghị chèn ảnh dialog chọn tài khoản Google + ảnh sau khi đăng nhập thành công.

3.7. Đăng nhập bằng Facebook

Tương tự Google, đăng nhập Facebook cũng sử dụng OAuth Credential nạp vào `FirebaseAuth`. Phần thiết kế đã được nhóm chuẩn bị, các bước thực hiện gồm:

24. Đăng ký App trên Meta for Developers (<https://developers.facebook.com>), khai báo Android Package Name và Key Hash.
25. Thêm dependency `com.facebook.android:facebook-login`.
26. Khai báo Facebook App ID và Client Token trong `res/values/strings.xml` và `AndroidManifest.xml` (theo hướng dẫn Meta).
27. Tạo `CallbackManager`, gắn vào nút `LoginButton` hoặc gọi `LoginManager.getInstance().loginWithReadPermissions(activity, listOf("email", "public_profile"))`.
28. Khi callback `onSuccess(LoginResult)` → lấy `AccessToken`, tạo `FacebookAuthProvider.getCredential(token.getToken())` →
`FirebaseAuth.signInWithCredential()`.
29. Lưu user vào Firestore với `loginType = FACEBOOK`.

Tại thời điểm hoàn thiện đề tài, phiên Facebook Login đang chờ Meta xét duyệt App từ trạng thái Development sang Live. Vì vậy nút “Đăng nhập với Facebook” đã được thiết kế sẵn trên giao diện nhưng được ẩn (`visibility=GONE`) cho đến khi App được duyệt. Khi đó, nhóm chỉ cần kích hoạt `visibility` và bổ sung dependency là tính

năng hoạt động đầy đủ. Kiến trúc này giúp Facebook Login có thể bật/tắt bằng feature flag mà không phải sửa logic chính.

3.8. Quản lý hồ sơ cá nhân và số địa chỉ

Sau khi đăng nhập, người dùng có thể vào ProfileFragment để xem thông tin cá nhân, tổng số đơn đã đặt và tổng số tiền đã chi. Nhấn “Chỉnh sửa hồ sơ” mở EditProfileActivity cho phép sửa: avatar (chọn ảnh từ thư viện → upload Firebase Storage → cập nhật avatarUrl), họ tên, số điện thoại, ngày sinh, giới tính. Cập nhật được lưu xuống Firestore và đồng thời cập nhật SessionManager (cache cục bộ).

Số địa chỉ được quản lý trong ManageAddressActivity. Các thao tác:

- Thêm địa chỉ mới qua dialog dialog_add_edit_address.xml.
- Sửa hoặc xóa địa chỉ; xóa có dialog xác nhận để tránh lỡ tay.
- Đặt một địa chỉ làm mặc định; địa chỉ mặc định sẽ được tự chọn ở Checkout.

3.9. Hệ thống thông báo (Notification)

Notification trong ứng dụng được thiết kế in-app dựa trên Firestore (chứa dùng FCM ở phiên bản hiện tại). Có ba loại thông báo:

- ORDER: phát sinh khi trạng thái đơn hàng thay đổi (Đã xác nhận, Đang giao, Đã giao, Đã hủy, Đã hoàn tiền...).
- PROMO: phát sinh khi admin tạo voucher hoặc bật một voucher đang vô hiệu lên (gửi cho TẤT CẢ khách hàng ACTIVE).
- SYSTEM: thông báo hệ thống chung.

Khi mở app, HomeFragment đăng ký NotificationRepository.listenUnreadCount() để hiển thị badge số thông báo chưa đọc theo thời gian thực. Click badge → mở NotificationActivity, tự động markAllAsRead trên Firestore. Click vào notification kiểu ORDER → điều hướng tới OrderDetailActivity tương ứng. Đoạn mã đếm chưa đọc:

```
public ListenerRegistration listenUnreadCount(String userId,
Callback<Long> cb) {
    return db.collection("notifications")
        .whereEqualTo("userId", userId)
        .addSnapshotListener((snap, e) -> {
            if (e != null) { cb.onSuccess(0L); return; }
            long unread = 0;
            if (snap != null) {
```

```

        for (DocumentSnapshot d : snap.getDocuments()) {
            Boolean isRead = d.getBoolean("isRead");
            if (isRead == null || !isRead) unread++;
        }
    }
    cb.onSuccess(unread);
});
}

```

3.10. Quản lý phiên đăng nhập (SessionManager, SharedPreferences)

Để giảm số lần truy vấn Firestore và xử lý nhanh trên client, nhóm thiết kế lớp SessionManager (singleton) bọc SharedPreferences với các thao tác: saveSession, updateUserName, updateUserAvatar, clearSession, getRole, isLoggedIn, isAdmin... SessionManager được khởi tạo ngay sau khi đăng nhập thành công và bị xoá khi logout.

Đoạn mã rút gọn của SessionManager:

```

public class SessionManager {
    private static final String PREF_NAME = "petshop_prefs";
    private final SharedPreferences prefs;
    private static SessionManager instance;

    public static SessionManager getInstance(Context context) {
        if (instance == null) instance = new SessionManager(context);
        return instance;
    }
    private SessionManager(Context context) {
        prefs = context.getApplicationContext()
            .getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE);
    }
    public void saveSession(String userId, String name, String email,
        String role, String avatar) {
        prefs.edit().putString("user_id", userId).putString("user_name",
name)
            .putString("user_email", email).putString("user_role",
role)
            .putString("user_avatar", avatar).apply();
    }
    public boolean isLoggedIn() { return prefs.getString("user_id",
null) != null; }
    public boolean isAdmin() { return
"ADMIN".equals(prefs.getString("user_role", "CUSTOMER")); }
    public void clearSession() { prefs.edit().clear().apply(); }
}

```

3.11. Kết quả đạt được và minh họa giao diện

Module Tài khoản đã được hoàn thiện 100% theo đặc tả ban đầu. Đăng ký, đăng nhập email và Google chạy ổn định trên cả emulator API 30/33/34 và thiết bị thật. OTP qua email đến trong vòng 5–10 giây. SessionManager hoạt động chính xác, đăng xuất xoá sạch dữ liệu cục bộ. Số địa chỉ và Notification đáp ứng các test case đặt ra (xem Chương 7).

[CHỤP ẢNH 3.11] Đề nghị chèn ảnh các màn hình tiêu biểu của module: Splash, Login, Register, Profile, Edit Profile, Manage Address, Notification (có badge và khi mở danh sách).

CHƯƠNG 4. XÂY DỰNG MODULE DANH MỤC SẢN PHẨM VÀ TRANG CHỦ

Thành viên phụ trách: TV2 – Đào Trúc Mai.

4.1. Phân tích chức năng dành cho khách hàng

Sau khi đăng nhập (hoặc duyệt ở chế độ guest), khách hàng tương tác chủ yếu với module Danh mục sản phẩm và Trang chủ. Đây là “mặt tiền” của ứng dụng – nơi quyết định trải nghiệm đầu tiên và tỉ lệ chuyển đổi sang đặt hàng. Vì vậy module được giao cho TV2 với yêu cầu cao về thẩm mỹ giao diện và tốc độ tải dữ liệu.

Các tính năng chính của module:

- Trang chủ (HomeFragment): banner ViewPager2, lời chào theo thời gian, danh mục, sản phẩm nổi bật.
- Tìm kiếm thời gian thực theo từ khoá.
- Lọc sản phẩm theo danh mục (Pet/Food).
- Trang Danh mục đầy đủ (CategoryFragment / ProductListActivity).
- Trang chi tiết Pet (PetDetailActivity) – ảnh slider, giống, tuổi, cân nặng, lịch sử tiêm phòng.
- Trang chi tiết Food (FoodDetailActivity) – ảnh, thương hiệu, loại, trọng lượng, hướng dẫn cho ăn.
- Hiển thị giá khuyến mãi (qua PromotionManager).
- Trang Khuyến mãi (PromotionActivity) hiển thị voucher hệ thống.

4.2. Thiết kế lớp Model: Pet, Food, Category, Banner, Favorite, Review

Bảng 4.1 và 4.2 tóm tắt các thuộc tính chính của hai entity quan trọng nhất – Pet và Food. Các trường còn lại (Category, Banner, Favorite, Review) được thiết kế đơn giản hơn và mô tả trong Phụ lục B.

Bảng 10: Các thuộc tính của Model Pet

Trường	Kiểu	Mô tả
id	String	ID duy nhất.
name	String	Tên thú cưng (ví dụ: 'Lulu').
categoryId	String	Khoá ngoại tới categories.

species	String	Loài (DOG, CAT, FISH, BIRD,...).
breed	String	Giống (Corgi, Husky, Persian, ...).
age	int	Tuổi.
ageUnit	String	MONTH YEAR.
gender	String	MALE FEMALE UNKNOWN.
weight	double	Cân nặng (kg).
color	String	Màu lông.
origin	String	Xuất xứ.
vaccineStatus	String	FULL PARTIAL NONE.
price	double	Giá gốc.
status	String	AVAILABLE RESERVED SOLD INACTIVE.
media	List	Danh sách PetMedia (ảnh + video).
description	String	Mô tả chi tiết.
createdAt/updatedAt	String	Mốc thời gian.

Bảng 11: Các thuộc tính của Model Food

Trường	Kiểu	Mô tả
id	String	ID duy nhất.
name	String	Tên sản phẩm.
categoryId	String	Khoá ngoại tới categories.
brand	String	Thương hiệu (Royal Canin, Whiskas,...).
foodType	String	DRY WET SNACK MILK SUPPLEMENT.
targetPetType	String	DOG CAT FISH BIRD RABBIT ALL.
weightGram	int	Trọng lượng gói (gram).
price	double	Giá gốc.
stock	int	Tồn kho hiện tại.
sold	int	Số lượng đã bán.
status	String	AVAILABLE OUT_OF_STOCK INACTIVE.
media	List	FoodMedia (ảnh).
description	String	Mô tả chi tiết.

4.3. Tổ chức Repository: FoodRepository, PetRepository, CategoryRepository

Cả ba repository đều dùng `Firestore.getInstance()` và cung cấp các phương thức: `getAll`, `getById`, `getByCategory`, `search`, `add` (chỉ admin), `update` (chỉ admin), `delete` (chỉ admin). Riêng `FoodRepository` có thêm `getStock` và `updateStock` để hỗ trợ luồng đặt hàng (kiểm tra/giảm tồn kho khi tạo đơn).

Mẫu callback chuẩn được sử dụng cho mọi repository:

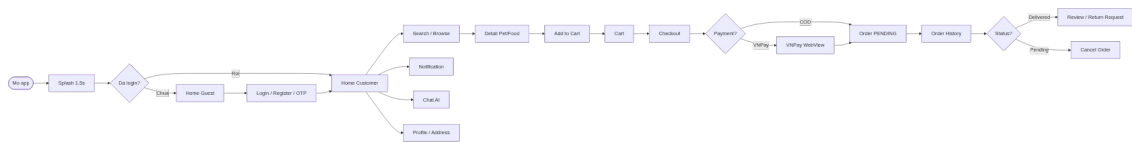
```
public interface Callback<T> {  
    void onSuccess(T data);  
    void onFailure(String error);  
}
```

Cách triển khai callback giúp các ViewModel xử lý kết quả/error nhất quán, đồng thời dễ chuyển sang Coroutines hoặc RxJava trong tương lai mà không phải đổi nhiều API.

4.4. Trang chủ (HomeFragment) – banner, danh mục, sản phẩm nổi bật

`HomeFragment` là Fragment đầu tiên hiển thị khi vào `PetShopActivity`. Layout được chia thành các khối từ trên xuống dưới:

- Top bar: lời chào (ví dụ “Hi, Mai 🐾”) thay đổi theo thời gian (sáng/chiều/tối) lấy từ `Calendar.HOUR_OF_DAY`; nút giỏ hàng (kèm badge số) và nút thông báo (badge số chưa đọc).
- Search bar realtime: `TextWatcher` gọi `vm.search(s.toString())` → `HomeViewModel` filter danh sách Pet/Food trong RAM rồi cập nhật lại `RecyclerView`.
- Banner `ViewPager2`: `ImageSliderAdapter` hiển thị các banner promo lấy từ collection banners. Tự động auto-scroll mỗi 4s.
- Danh mục (`RecyclerView` ngang): `HomeCategoryAdapter`; click vào danh mục sẽ filter sản phẩm tương ứng.
- Section “Thú cưng nổi bật” (RV ngang) + “Thức ăn nổi bật” (RV ngang) với nút “Xem tất cả” mở `ProductListActivity`.
- Card Promo: dẫn sang `PromotionActivity`.



Hình 12: Hành trình khách hàng – HomeFragment là điểm xuất phát

[CHỤP ẢNH 4.4] Đề nghị chèn ảnh đầy đủ HomeFragment (1 ảnh).

4.5. Màn hình Danh mục (CategoryFragment / ProductListActivity)

Khi người dùng nhấn “Xem tất cả” từ Trang chủ hoặc chọn một danh mục cụ thể, ứng dụng mở ProductListActivity với extra EXTRA_CATEGORY = PET hoặc FOOD và (tùy chọn) EXTRA_FILTER_KEY (tên danh mục cụ thể). Activity hiển thị danh sách sản phẩm dạng lưới 2 cột với khả năng:

- Sắp xếp theo: Mới nhất, Giá tăng dần, Giá giảm dần, Bán chạy.
- Lọc theo loài/loại thức ăn, theo giá (slider), theo có khuyến mãi hay không.
- Vô hạn cuộn (đang cài đặt phân trang ở phiên bản nâng cao).

[CHỤP ẢNH 4.5] Đề nghị chèn ảnh ProductListActivity với các tùy chọn lọc.

4.6. Màn hình Chi tiết thú cưng (PetDetailActivity)

PetDetailActivity hiển thị thông tin chi tiết của một cá thể thú cưng: ảnh slider (kết hợp video nếu có), tên, giống, tuổi (đổi đơn vị tự động giữa MONTH/YEAR), cân nặng, giới tính, màu lông, xuất xứ, tình trạng tiêm phòng (chip màu xanh/vàng/đỏ tương ứng FULL/PARTIAL/NONE), giá gốc – giá khuyến mãi (nếu có), nút “Thêm vào giỏ” (chỉ kích hoạt khi status == AVAILABLE).

Khác biệt quan trọng so với thương mại điện tử thông thường: mỗi pet là một cá thể duy nhất, vì vậy số lượng luôn là 1. Khi pet được thêm vào giỏ, status chuyển sang RESERVED để các khách khác không thể đặt cùng lúc; khi đơn bị hủy, status sẽ tự động trả lại AVAILABLE thông qua transaction trong OrderRepository.

[CHỤP ẢNH 4.6] Đề nghị chèn ảnh PetDetailActivity (1 ảnh có khuyến mãi + 1 ảnh đã RESERVED).

4.7. Màn hình Chi tiết thức ăn (FoodDetailActivity)

FoodDetailActivity tương tự PetDetail nhưng có các điểm khác:

- Có bộ chọn số lượng (+/–), giới hạn theo stock hiện tại.

- Hiển thị thương hiệu, loại thức ăn, đối tượng phù hợp (chó/mèo/cá...), trọng lượng/gói.
- Hiển thị tab “Mô tả”, “Thành phần”, “Hướng dẫn cho ăn”, “Đánh giá”.
- Nút Thêm vào giỏ trừ trực tiếp số lượng vào tồn kho khi đơn được tạo (qua transaction).

[CHỤP ẢNH 4.7] Đề nghị chèn ảnh FoodDetailActivity.

4.8. Hiển thị hình ảnh với Glide và Firebase Storage

Toàn bộ ảnh trong ứng dụng (banner, ảnh sản phẩm, avatar, ảnh tin nhắn chat) được lưu trên Firebase Storage và truy cập qua URL. Để tải hiệu quả, nhóm dùng thư viện Glide với cú pháp:

```
Glide.with(context)
    .load(url)
    .placeholder(R.drawable.bg_image_placeholder)
    .error(R.drawable.bg_image_placeholder)
    .centerCrop()
    .into(imageView);
```

Glide tự động cache ảnh vào bộ nhớ và đĩa, hỗ trợ hiển thị placeholder khi đang tải, error khi tải lỗi, transformation (centerCrop, circleCrop) – rất phù hợp cho RecyclerView nhiều item.

4.9. Tìm kiếm và lọc sản phẩm

Tìm kiếm trong ứng dụng được thực hiện theo hai tầng: (1) tìm kiếm cục bộ trong RAM trên dữ liệu đã tải về từ HomeViewModel/ProductViewModel – cho tốc độ phản hồi tức thời; (2) khi danh sách quá lớn, có thể chuyển sang truy vấn Firestore với `.whereGreaterThanOrEqualTo("name", q).whereLessThanOrEqualTo("name", q + "\uffff")`. Phiên bản hiện tại sử dụng tầng (1) là chính.

Lọc theo danh mục, lọc theo loại thức ăn, lọc theo khoảng giá đều thực hiện bằng cách áp predicate lên `List<Pet>` / `List<Food>` rồi `postValue` để LiveData cập nhật RecyclerView. Cách làm này đơn giản, dễ mở rộng và không phát sinh chi phí read Firestore.

4.10. Kết quả đạt được và minh họa giao diện

Module Trang chủ và Danh mục được hoàn thiện đầy đủ với UI mượt, ảnh tải nhanh nhờ Glide, banner tự động chạy. Tìm kiếm phản hồi tức thời (~50 ms) cho danh sách 200 sản phẩm. Hiển thị giá khuyến mãi và badge phù hợp. Pet RESERVED được phân biệt rõ với pet AVAILABLE.

[CHỤP ẢNH 4.10] Đề nghị chèn 4 ảnh: Trang chủ đầy đủ, kết quả tìm kiếm, ProductList, Pet Detail / Food Detail.

CHƯƠNG 5. XÂY DỰNG MODULE GIỎ HÀNG, THANH TOÁN VÀ ĐƠN HÀNG

Thành viên phụ trách: TV3 – Nguyễn Văn Trường.

5.1. Phân tích chức năng mua hàng

Module Giỏ hàng – Thanh toán – Đơn hàng là phần phức tạp nhất của ứng dụng vì liên quan đến giao dịch tiền bạc, tồn kho, voucher và tích hợp với cổng thanh toán bên thứ ba. Module được phân công cho TV3 vì các yêu cầu sau:

- Phải đảm bảo tính nguyên tử (atomic) khi tạo đơn: không được trừ stock 2 lần, không cho phép 2 khách cùng mua một con pet.
- Phải xử lý đầy đủ trạng thái thanh toán (COD vs VNPay) và phải đồng bộ tồn kho khi thanh toán thành công/thất bại.
- Phải áp dụng đúng voucher/promotion với nhiều loại (PERCENT, FIXED, FREESHIP).
- Phải xử lý vòng đời đơn hàng đầy đủ: từ tạo, theo dõi đến huỷ, hoàn trả, hoàn tiền.

5.2. Thiết kế lớp Model

Module sử dụng các lớp Model: Cart, CartItem, Order, OrderItem, ReturnRequest, PaymentTransaction. Quan hệ giữa chúng:

- Cart (1) – CartItem (N): mỗi user có một Cart (id = uid), chứa danh sách CartItem.
- Order (1) – OrderItem (N): khi tạo đơn, các CartItem được copy thành OrderItem để cố định giá tại thời điểm mua.
- Order (1) – ReturnRequest (0..1): mỗi đơn có thể có tối đa một yêu cầu hoàn trả đang xử lý.
- Order (1) – PaymentTransaction (0..N): lịch sử các giao dịch VNPay liên quan tới đơn (thử lại, refund...).

5.3. Quản lý giỏ hàng (CartFragment / CartActivity)

Giỏ hàng có hai điểm vào: nút giỏ trên Top bar (mở CartActivity) và tab Cart trong bottom navigation (mở CartFragment). Cả hai cùng dùng CartViewModel để giữ trạng thái nhất quán.

Các thao tác chính:

- Hiển thị danh sách CartItem; mỗi item có ảnh, tên, giá, số lượng và nút xoá.
- Cộng/trừ số lượng (chỉ áp dụng cho Food) – có kiểm tra stock thông qua FoodRepository.getStock().
- Xoá item: dialog xác nhận; nếu là Pet thì khôi phục status về AVAILABLE.
- Tổng tiền tạm tính: hiển thị real-time qua LiveData.
- Nút “Thanh toán” mở CheckoutActivity nếu giỏ hàng không trống.

Đoạn mã thêm Pet vào giỏ (rút gọn từ CartRepository.addPetToCart):

```
db.runTransaction(tx -> {
    var petRef = db.collection("pets").document(pet.getId());
    var cartRef = db.collection("carts").document(userId);

    var petDoc = tx.get(petRef);
    var cartDoc = tx.get(cartRef);

    String status = petDoc.getString("status");
    if (!"AVAILABLE".equals(status))
        throw new RuntimeException("Thu cung nay da duoc dat boi nguoi
khac");

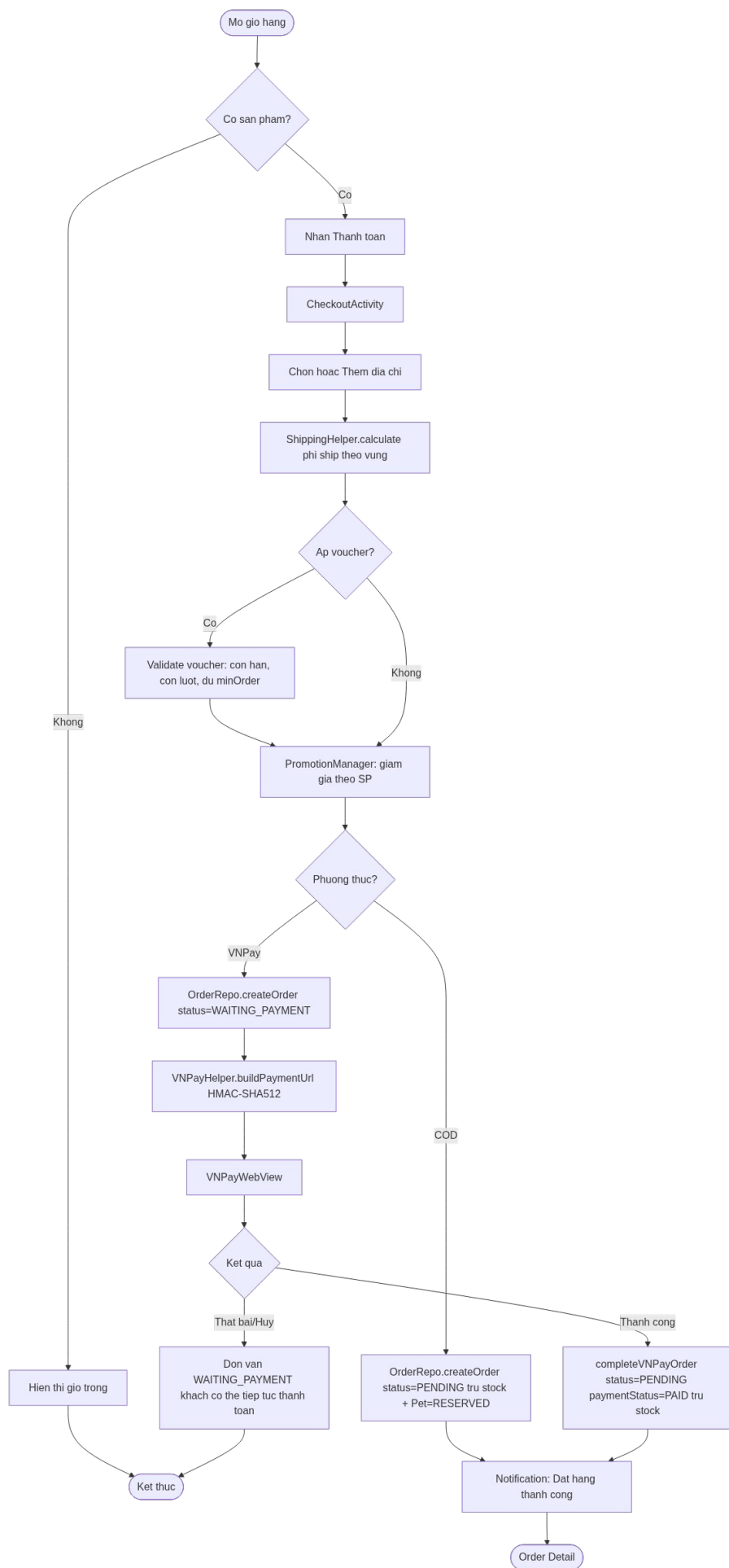
    tx.update(petRef, "status", "RESERVED");

    Cart cart = cartDoc.exists() ? cartDoc.toObject(Cart.class) : new
Cart(userId);
    cart.addItem(new CartItem(...));
    tx.set(cartRef, cart);
    return null;
});
```

Việc thực hiện trong runTransaction giúp ngăn chặn race condition khi 2 khách hàng cùng nhấn “Thêm vào giỏ” vào cùng một con pet.

5.4. Quy trình thanh toán (CheckoutActivity)

CheckoutActivity là màn hình tổng hợp toàn bộ thông tin đơn hàng trước khi gửi đi thanh toán. Sơ đồ luồng được trình bày trong Hình 5.1.



Hình 13: Activity Diagram – luồng đặt hàng & thanh toán

5.4.1. Chọn địa chỉ giao hàng

Phần đầu màn hình hiển thị địa chỉ mặc định của user. Người dùng có thể nhấn “Thay đổi” để mở ManageAddressActivity và chọn một địa chỉ khác. Sau khi chọn, CheckoutActivity sẽ tự tính lại phí ship và cập nhật tổng tiền.

5.4.2. Áp dụng Voucher / Promotion

Module hỗ trợ ba loại voucher (Voucher) và promotion tự động (Promotion). Khi user mở CheckoutActivity:

- Hệ thống tự động tải các voucher còn hiệu lực (VoucherRepository.getSystemVouchers) hiển thị dạng chip để user chọn.
- Hệ thống tải các promotion AUTOMATIC còn hạn (PromotionRepository.getActive) và áp dụng vào giá sản phẩm thông qua PromotionManager.refreshCartPrices.
- Khi user chọn voucher: VoucherRepository.getByCode để xác thực; kiểm tra điều kiện (đơn tối thiểu, lượt còn lại, số lần đã dùng của user qua collection voucher_usage).
- Voucher PERCENT giới hạn bởi maxDiscountAmount; FIXED giảm cố định; FREESHIP đặt phí ship = 0.

5.4.3. Tính phí vận chuyển (ShippingHelper)

Phí ship được tính trong ShippingHelper dựa trên thành phố nhận hàng và tổng tiền sản phẩm. Bảng 5.1 mô tả chính sách hiện tại.

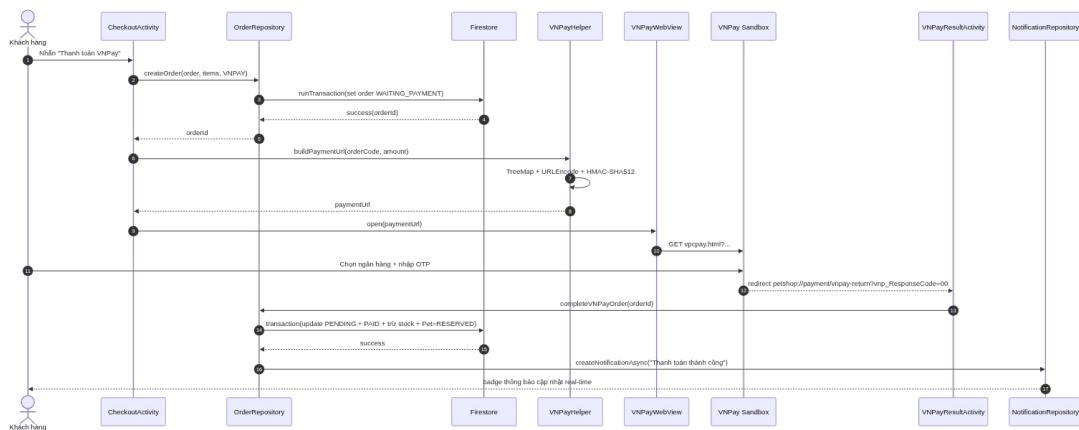
Bảng 12: Bảng giá phí vận chuyển theo vùng

Khu vực	Phí ship	Thời gian dự kiến
TP. Hồ Chí Minh (cùng thành phố)	30.000đ	1–2 ngày
Miền Nam khác (Bình Dương, Đồng Nai, Cần Thơ, ...)	45.000đ	2–3 ngày
Miền Trung (Đà Nẵng, Huế, Quảng Nam, Khánh Hoà, ...)	60.000đ	3–4 ngày
Miền Bắc (Hà Nội, Hải Phòng, ...)	75.000đ	4–6 ngày
Đơn ≥ 500.000đ (mọi vùng)	MIỄN PHÍ	Như trên

Logic tính phí được đóng gói trong `ShippingHelper.calculate(address, subtotal, callback)`. Trong tương lai, lớp này có thể được thay bằng client gọi API GHN/GHTK/J&T mà không ảnh hưởng các phần khác của hệ thống.

5.5. Tích hợp cổng thanh toán VNPay

Tích hợp VNPay được thực hiện qua ba lớp: `VNPayHelper` (build URL + chữ ký), `VNPayWebViewActivity` (mở WebView tới sandbox), `VNPayResultActivity` (xử lý callback Deep Link). Sơ đồ tuần tự thể hiện rõ tương tác giữa các lớp:



Hình 14: Sequence Diagram – thanh toán qua VNPay



Hình 15: Sơ đồ luồng dữ liệu thanh toán VNPay

5.5.1. Tạo URL thanh toán (VNPayHelper)

Theo tài liệu chính thức của VNPay, URL thanh toán phải có chữ ký HMAC-SHA512 tính từ chuỗi tham số đã URL-encode UTF-8 và sort theo tên trường. Bảng 5.2 mô tả các tham số tiêu biểu.

Bảng 13: Tham số chính của VNPay buildPaymentUrl

Tham số	Mô tả	Ví dụ
vnp_Version	Phiên bản API	2.1.0
vnp_Command	Loại lệnh	pay
vnp_TmnCode	Mã website ký hợp đồng với VNPay	ABCDXXXX
vnp_Amount	Số tiền (đơn vị 1 = 0.01 VND)	1500000 (= 15.000đ)
vnp_CurrCode	Tiền tệ	VND
vnp_TxnRef	Mã đơn hàng (duy nhất)	ORD0531123456ABCDEF
vnp_OrderInfo	Thông tin đơn	Thanh toan don ORD...
vnp_Locale	Ngôn ngữ giao diện VNPay	vn
vnp_ReturnUrl	URL/Deep Link sau thanh toán	petshop://payment/vnpay-

		return
vnp_CreateDate / vnp_ExpireDate	Thời điểm tạo / hết hạn (15 phút sau)	20260510052000 / 20260510053500
vnp_SecureHash	Chữ ký HMAC- SHA512(secret, hashData)	lowercase hex 128 ký tự

Cài đặt rút gọn của VNPayHelper.buildPaymentUrl():

```
Map<String, String> p = new TreeMap<>();
p.put("vnp_Version", "2.1.0");
p.put("vnp_Command", "pay");
p.put("vnp_TmnCode", Constants.VNPAY_TMN_CODE);
p.put("vnp_Amount", String.valueOf(amount * 100));
p.put("vnp_CurrCode", "VND");
p.put("vnp_TxnRef", orderCode);
p.put("vnp_OrderInfo", orderInfo);
p.put("vnp_OrderType", "other");
p.put("vnp_Locale", "vn");
p.put("vnp_ReturnUrl", Constants.VNPAY_RETURN_URL);
p.put("vnp_IpAddr", "127.0.0.1");
p.put("vnp_CreateDate", now);
p.put("vnp_ExpireDate", now15);

StringBuilder hashData = new StringBuilder();
StringBuilder query = new StringBuilder();
for (String name : new ArrayList<>(p.keySet())) {
    String value = p.get(name);
    if (value == null || value.isEmpty()) continue;
    String en = URLEncoder.encode(name, "UTF-8");
    String ev = URLEncoder.encode(value, "UTF-8");
    if (hashData.length() > 0) { hashData.append('&'); }
    query.append('&'); }
    hashData.append(en).append('=').append(ev);
    query.append(en).append('=').append(ev);
}
String secureHash = hmacSHA512(Constants.VNPAY_HASH_SECRET,
hashData.toString());
return Constants.VNPAY_URL + "?" + query + "&vnp_SecureHash=" +
secureHash;
```

5.5.2. *Hiển thị WebView (VNPayWebViewActivity)*

VNPayWebViewActivity nạp URL trên WebView và override shouldOverrideUrlLoading() để bắt mọi redirect. Khi VNPay gửi user về Deep Link petshop://payment/vnpay-return?vnp_ResponseCode=..., WebView nhận URL nhưng không thể tự xử lý → ứng dụng chuyển sang Intent với scheme petshop:// và Android sẽ launch VNPayResultActivity.

5.5.3. Xử lý kết quả và Deep Link (VNPayResultActivity)

VNPayResultActivity là một Activity exported = true với intent-filter:

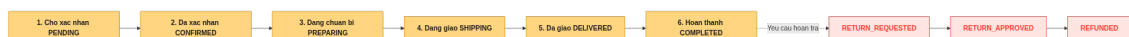
```
<activity android:name=".view.activity.VNPayResultActivity"
android:exported="true">
    <intent-filter>
        <action android:name="android.intent.action.VIEW" />
        <category android:name="android.intent.category.DEFAULT" />
        <category android:name="android.intent.category.BROWSABLE" />
        <data android:scheme="petshop" android:host="payment"
android:path="/vnpay-return" />
        <data android:scheme="https" android:host="petshop-
payment.web.app"
android:path="/vnpay-return" />
    </intent-filter>
</activity>
```

Activity đọc các query param vnp_ResponseCode và vnp_TxnRef từ Intent. Nếu vnp_ResponseCode == "00" (thành công), gọi OrderRepository.completeVNPayOrder() trong một transaction để: cập nhật status = PENDING, paymentStatus = PAID, trừ stock food, đặt Pet = RESERVED, đồng thời lưu PaymentTransaction. Cuối cùng gửi notification “Thanh toán thành công” cho user.

5.6. Quản lý đơn hàng (OrderHistory, OrderDetail)

OrderHistoryActivity hiển thị danh sách đơn của user, có tab lọc theo trạng thái: Chờ xử lý, Đang giao, Đã giao, Đã hủy, Hoàn tiền. OrderDetailActivity hiển thị chi tiết một đơn cùng các nút thao tác:

- Hủy đơn: chỉ khi status $\in$ {PENDING, WAITING_PAYMENT, CONFIRMED}; hoàn lại stock + voucher.
- Đánh giá: chỉ khi status $\in$ {DELIVERED, COMPLETED}.
- Yêu cầu hoàn trả: như Đánh giá.
- Mua lại: tạo đơn mới với cùng danh sách item (nếu còn).

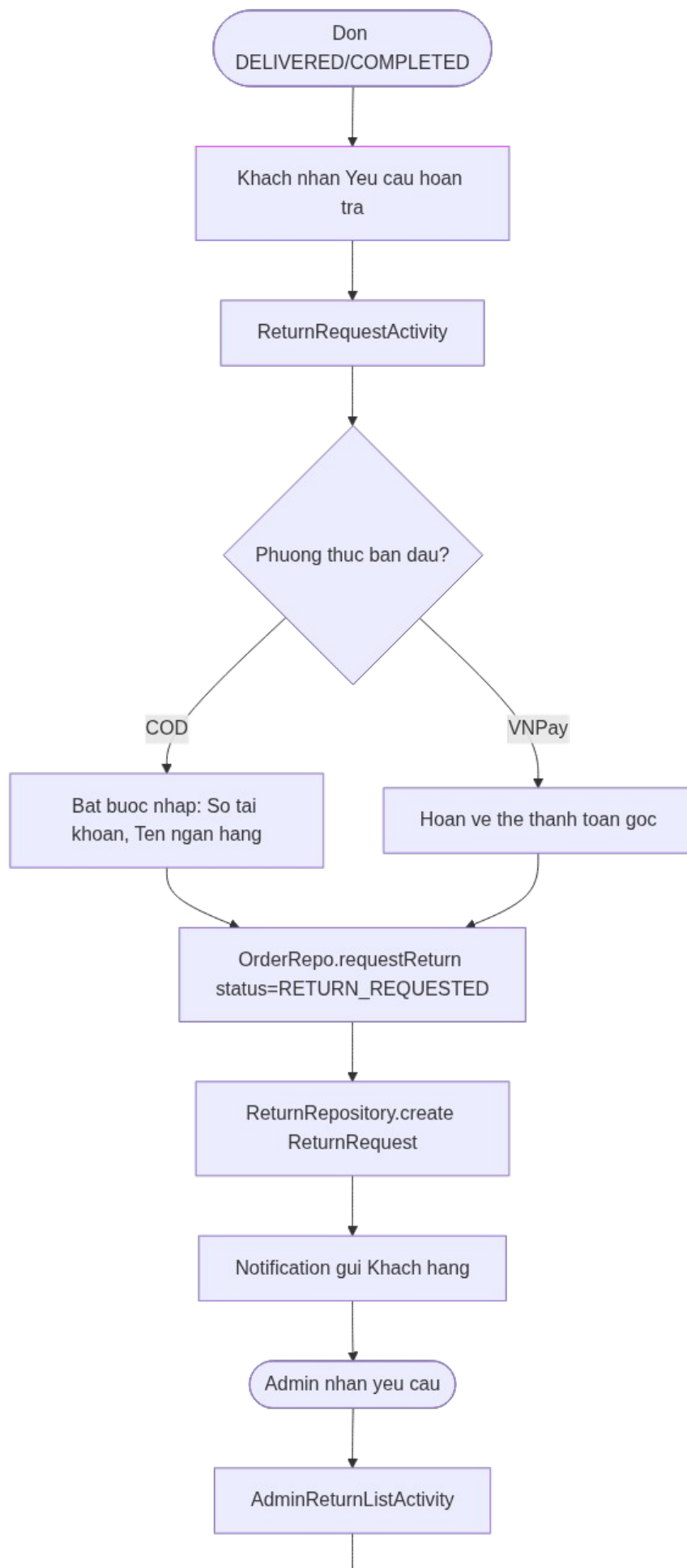


Hình 16: Sơ đồ trạng thái thanh đơn hàng hiển thị cho khách hàng

5.7. Yêu cầu trả hàng / hoàn tiền (ReturnRequestActivity)

Khi user yêu cầu hoàn trả, ReturnRequestActivity yêu cầu nhập lý do và (nếu thanh toán COD) nhập số tài khoản + tên ngân hàng để admin chuyển khoản. Sau khi

gửi, đơn chuyển sang RETURN_REQUESTED và xuất hiện trong danh sách yêu cầu của admin (xem Chương 6).



Hình 17: Activity Diagram – yêu cầu trả hàng và hoàn tiền (chi tiết)

5.8. Kết quả đạt được và minh họa giao diện

Module Giỏ hàng – Thanh toán – Đơn hàng đã được hoàn thiện đầy đủ. Các test case quan trọng đều pass: tạo đơn COD/VNPay, huỷ đơn pending, áp voucher, hoàn trả COD và VNPay. VNPay sandbox xử lý trong vòng 5–8 giây cho mỗi giao dịch. Việc chuyển trạng thái đơn được thực hiện qua Firestore Transaction đảm bảo tính nhất quán.

[CHỤP ẢNH 5.8] Đề nghị chèn các ảnh: Cart, Checkout (chọn voucher), VNPay sandbox (chọn ngân hàng + nhập OTP NCB), VNPay Result thành công, Order History (đa trạng thái), OrderDetail, ReturnRequest.

CHƯƠNG 6. XÂY DỰNG MODULE QUẢN TRỊ VÀ CHATBOT TƯ VẤN

Thành viên phụ trách: TV4 –

6.1. Phân tích chức năng cho vai trò Admin

Module Quản trị và Chatbot tư vấn là phần “sau hậu trường” của ứng dụng – nơi quyết định toàn bộ dữ liệu hiển thị cho khách hàng. Vai trò Admin bao gồm:

- Theo dõi tình hình kinh doanh real-time qua Dashboard.
- Quản lý người dùng: khoá/mở khoá, đổi role.
- Quản lý catalog: danh mục, thú cưng, thức ăn.
- Quản lý marketing: voucher, khuyến mãi.
- Quản lý vận hành: đơn hàng, yêu cầu trả hàng.
- Trả lời tư vấn khách hàng (qua hệ thống Chatbot AI tự động + có thể can thiệp thủ công nếu cần).

6.2. Phân quyền Admin (AdminSetupHelper)

Vì Firebase Authentication không có khái niệm role tự thân, nhóm thiết kế lớp AdminSetupHelper để khởi tạo tài khoản admin đầu tiên. Lớp này chỉ chạy 1 lần để bootstrap, sau đó nên xoá file để bảo mật. Quy trình:

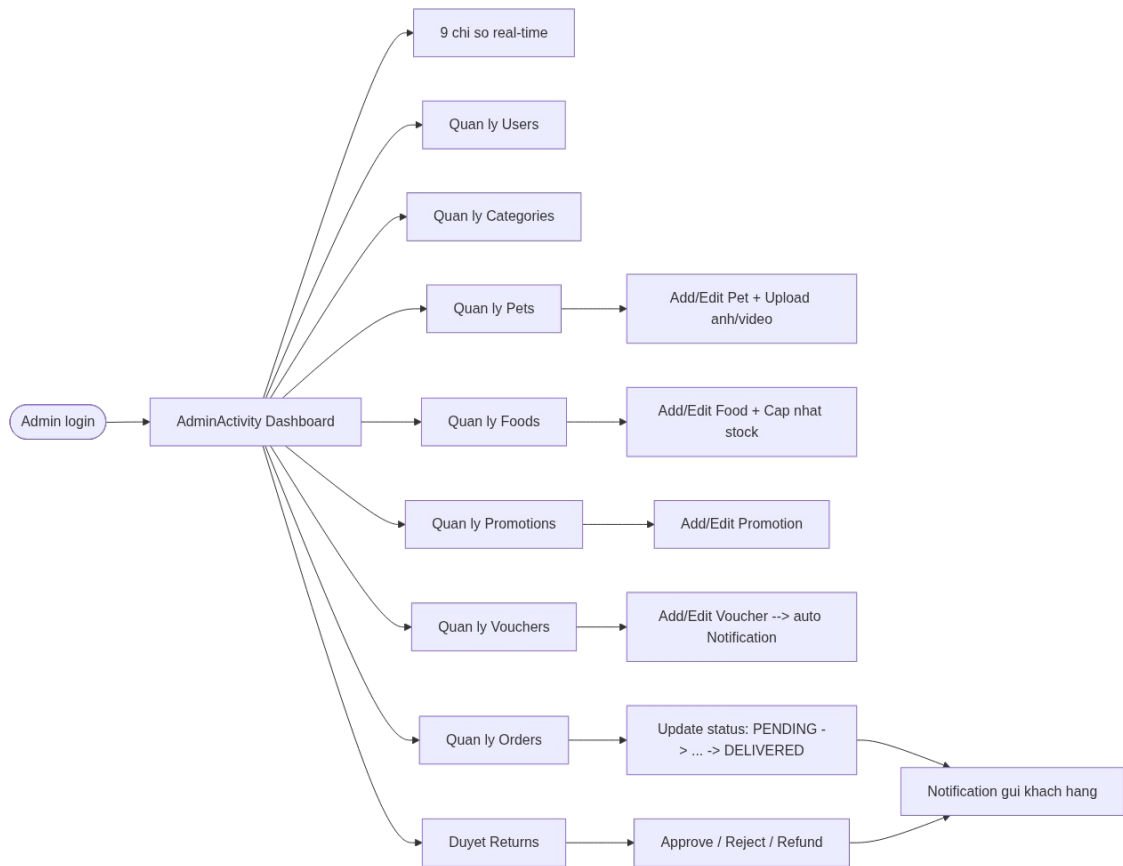
30. Nhập email và mật khẩu trong AdminSetupHelper.createAdminAccount().
31. Hệ thống gọi FirebaseAuth.createUserWithEmailAndPassword().
32. Sau khi tạo, lưu user vào Firestore với role = ADMIN.
33. Nếu email đã tồn tại trong Auth → tự động chuyển sang upgradeExistingToAdmin (chỉ update role trong Firestore).

Sau khi có tài khoản admin gốc, các admin khác có thể được “bổ nhiệm” bằng cách admin gốc đổi role của user trong ManageUsersActivity. Việc kiểm soát quyền vẫn dựa trên Firestore Security Rules (xem 2.4.3).

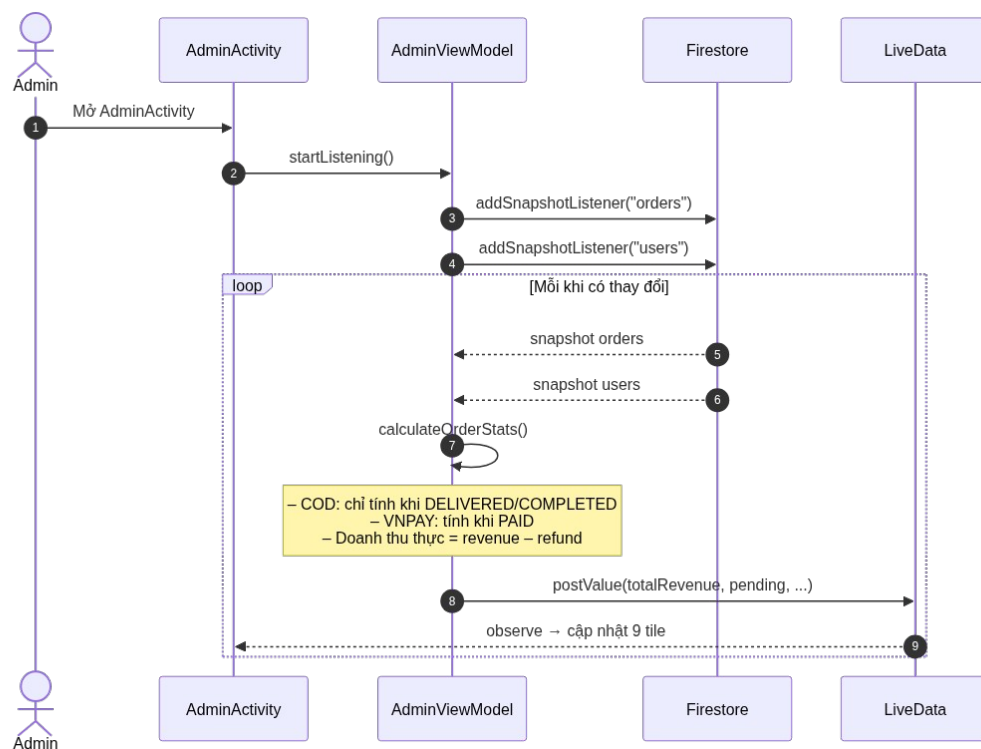
6.3. Trang điều khiển AdminActivity

AdminActivity là “trung tâm chỉ huy”. Layout có:

- Drawer bên trái với các mục: Dashboard, Người dùng, Danh mục, Thú cưng, Thức ăn, Khuyến mãi, Voucher, Đơn hàng, Hoàn trả, Đăng xuất.
- Phần chính hiển thị 9 thẻ thống kê real-time (Hình 6.1).
- AdminViewModel đăng ký hai SnapshotListener (orders + users) ngay khi mở Activity và hủy khi đóng để tránh leak.



Hình 18: Hành trình quản trị viên (Admin Journey)



Hình 19: Sequence Diagram – Admin Dashboard real-time

Bảng 14: Các tab quản trị trong AdminActivity

Tab	Activity tương ứng	Chức năng chính
Dashboard	AdminActivity	9 thẻ thống kê real-time.
Người dùng	ManageUsersActivity	Tìm kiếm, lọc role, khoá/mở khoá tài khoản.
Danh mục	ManageCategoriesActivity	CRUD danh mục Pet/Food.
Thú cưng	ManagePetsActivity + AddEditPetActivity	CRUD pet, upload ảnh + video.
Thức ăn	ManageFoodActivity + AddEditFoodActivity	CRUD food, dialog cập nhật stock.
Khuyến mãi	ManagePromotionsActivity + AddEditPromotionActivity	CRUD promotion tự động.
Voucher	ManageVouchersActivity + AddEditVoucherActivity	CRUD voucher; auto gửi notification khi tạo/bật.
Đơn hàng	AdminOrderListActivity + AdminOrderDetailActivity	Lọc, tìm kiếm, chuyển trạng thái đơn.
Hoàn trả	AdminReturnListActivity	Approve/Reject/Refund yêu cầu hoàn trả.

6.4. Quản lý người dùng (ManageUsersActivity)

Cho phép admin tìm kiếm theo tên/email, lọc theo role, khoá tài khoản (đặt status = BANNED) hoặc mở lại. Khi tài khoản bị BANNED, lần đăng nhập tiếp theo sẽ bị từ

chối với thông báo “Tài khoản đã bị khoá”. Việc khoá/mở khoá được lưu trên Firestore và đồng bộ tới các dashboard khác qua SnapshotListener.

6.5. Quản lý danh mục (ManageCategoriesActivity)

Admin có thể thêm/sửa/xoá danh mục Pet và danh mục Food (mỗi loại có color tag riêng để dễ phân biệt). Dialog `dialog_add_edit_category.xml` cho phép nhập tên và icon URL. Khi xoá, hệ thống cảnh báo nếu còn sản phẩm đang gắn vào danh mục.

6.6. Quản lý thú cưng (ManagePetsActivity / AddEditPetActivity)

ManagePetsActivity hiển thị danh sách pet với search, filter status. Mỗi item có nút Sửa/Xoá. Nhấn “Thêm” mở AddEditPetActivity – form khá phức tạp với các trường: tên, danh mục, loài, giống, tuổi, cân nặng, giới tính, màu, xuất xứ, tình trạng tiêm phòng, giá, mô tả, danh sách media (ảnh + video).

Phần upload media được TV4 thiết kế bằng MediaPickerAdapter cho phép chọn nhiều ảnh/video từ thư viện hoặc chụp/quay trực tiếp. Mỗi file được upload qua StorageHelper rồi thu URL trả về để lưu vào trường media của Pet.

6.7. Quản lý thức ăn (ManageFoodActivity / AddEditFoodActivity)

Tương tự ManagePets nhưng có thêm dialog cập nhật stock nhanh `dialog_update_stock.xml` – admin có thể cộng/trừ tồn kho mà không cần mở form chỉnh sửa toàn bộ. Khi stock = 0, status tự động chuyển OUT_OF_STOCK; khi stock > 0 trở lại, status quay về AVAILABLE.

6.8. Quản lý khuyến mãi (ManagePromotionsActivity)

Promotion là khuyến mãi tự động (không cần khách nhập mã). Admin định nghĩa: tên, mô tả, kiểu áp dụng (ALL/CATEGORY/PRODUCT), discountValue (PERCENT hoặc FIXED), maxDiscountAmount, startDate, endDate, isActive. Khi tải sản phẩm, PromotionManager.applyPromotions tự động chọn promotion tốt nhất cho từng sản phẩm và gắn vào trường discountedPrice.

6.9. Quản lý voucher (ManageVouchersActivity)

Voucher là mã giảm giá khách phải nhập trong Checkout. Có ba loại: PERCENT, FIXED, FREESHIP. Mỗi voucher có usageLimit (tổng) và perUserLimit (mỗi user).

Khi admin tạo hoặc bật một voucher, VoucherRepository tự động gửi notification kiểu PROMO cho TẤT CẢ user ACTIVE, ví dụ:

```
private void sendVoucherNotification(Voucher voucher) {
    db.collection("users")
        .whereEqualTo("role", "CUSTOMER")
        .whereEqualTo("status", "ACTIVE")
        .get()
        .addOnSuccessListener(snap -> {
            String title = "Voucher moi! 🎁";
            String message = buildVoucherMessage(voucher);
            for (var doc : snap.getDocuments()) {
                Notification n = new Notification();
                n.setUserId(doc.getId());
                n.setTitle(title); n.setMessage(message);
                n.setType("PROMO"); n.setRead(false);
                db.collection("notifications").document(...)
                    .set(n);
            }
        });
}
```

6.10. Quản lý đơn hàng và yêu cầu trả hàng phía Admin

AdminOrderListActivity cho phép admin lọc theo trạng thái, tìm kiếm theo mã đơn. AdminOrderDetailActivity cho phép cập nhật trạng thái đơn (ví dụ chuyển từ PENDING → CONFIRMED → PREPARING → SHIPPING → DELIVERED). Mỗi lần đổi trạng thái đều ghi log thời gian (paidAt, deliveredAt) và gửi notification cho khách. Nếu đơn chuyển sang DELIVERED hoặc COMPLETED, hệ thống tự cập nhật totalOrders, totalSpent của user (best-effort qua FieldValue.increment).

AdminReturnListActivity hiển thị các yêu cầu hoàn trả theo tab PENDING / APPROVED / REFUNDED / REJECTED. Admin xét duyệt và xử lý theo luồng đã mô tả ở Chương 5.

6.11. Tải ảnh sản phẩm lên Firebase Storage (StorageHelper)

StorageHelper là wrapper đơn giản quanh FirebaseStorage để upload ảnh/video và xoá file. Mỗi file được lưu tại đường dẫn folder/UUID.{ext} – đảm bảo không trùng tên. URL download được trả về dưới dạng String để lưu vào Firestore.

```
public static void uploadImage(Uri fileUri, String folder,
    OnUploadCallback cb) {
    String fileName = folder + "/" + UUID.randomUUID() + ".jpg";
    StorageReference ref = storage.getReference().child(fileName);
    ref.putFile(fileUri)
```

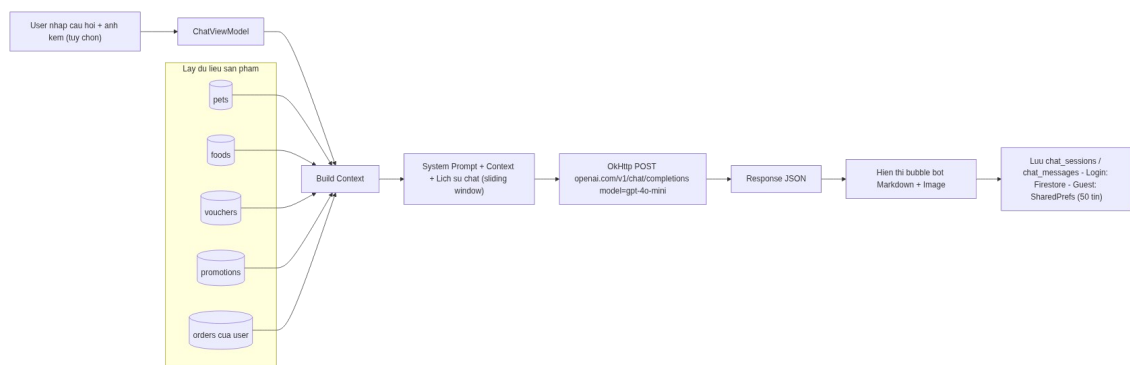
```

        .continueWithTask(task -> {
            if (!task.isSuccessful()) throw task.getException();
            return ref.getDownloadUrl();
        })
        .addOnSuccessListener(uri -> cb.onSuccess(uri.toString()))
        .addOnFailureListener(e -> cb.onFailure(e.getMessage()));
    }

```

6.12. Chatbot tư vấn dựa trên OpenAI API

Chatbot là điểm khác biệt nổi bật của ứng dụng so với các giải pháp truyền thống. Chatbot được xây dựng theo kiến trúc RAG (Retrieval-Augmented Generation) đơn giản:



Hình 20: Sơ đồ kiến trúc Chatbot RAG nhẹ

6.12.1. Thiết kế hội thoại

Chat của ứng dụng có:

- Welcome message: “Chào bạn! Tôi là trợ lý ảo của PetShop. Tôi có thể giúp gì cho bạn hôm nay?”.
- Cho phép gửi: văn bản, ảnh (Base64 inline), voice (chuyển thành text qua RecognizerIntent).
- Drawer bên phải hiển thị lịch sử các phiên hội thoại; nhấn vào phiên cũ sẽ tải tin nhắn cũ.
- Bot trả lời theo Markdown đơn giản (gạch đầu dòng, in đậm).

6.12.2. Gọi API bằng OkHttp + Gson

ChatViewModel sử dụng OkHttp để gọi POST tới <https://api.openai.com/v1/chat/completions> với model gpt-4o-mini. Trước khi gọi, ViewModel tự xây “system prompt” gồm: dữ liệu danh mục, danh sách Pet/Food còn bán, voucher đang chạy, đơn hàng của user. Đây là cách triển khai RAG đơn giản, không cần vector database.

```

JsonObject body = new JsonObject();
body.addProperty("model", "gpt-4o-mini");

JsonArray msgs = new JsonArray();
JsonObject sys = new JsonObject();
sys.addProperty("role", "system");
sys.addProperty("content",
    "Bạn là trợ lý ảo PetShop. " +
    "Dưới đây là dữ liệu hệ thống bạn được phép trả lời:\n" +
    userContext);
msgs.add(sys);

for (ChatMessage m : recentHistory) {
    JsonObject h = new JsonObject();
    h.addProperty("role", m.getType() == ChatMessage.TYPE_USER ?
        "user" : "assistant");
    h.addProperty("content", m.getContent());
    msgs.add(h);
}
body.add("messages", msgs);

Request req = new Request.Builder()
    .url("https://api.openai.com/v1/chat/completions")
    .addHeader("Authorization", "Bearer " + BuildConfig.OPENAI_API_KEY)
    .addHeader("Content-Type", "application/json")
    .post(RequestBody.create(body.toString(),
        MediaType.parse("application/json")))
    .build();
client.newCall(req).enqueue(...);

```

6.12.3. Lưu lịch sử trò chuyện trên Firestore

Để hỗ trợ truy cứu lịch sử và cá nhân hoá trải nghiệm, mọi tin nhắn đều được lưu lên Firestore theo cấu trúc `users/{uid}/sessions/{sessionId}/messages/{msgId}`. Khi user mở Chat lần sau, ChatViewModel tải session gần nhất và mở lại. Khách chưa đăng nhập (guest) lưu cục bộ trong SharedPreferences với giới hạn 50 tin để không phình app data.

6.13. Kết quả đạt được và minh hoạ giao diện

Toàn bộ module Quản trị đã hoàn thiện 9 tính năng. Dashboard cập nhật ngay tức thì khi có đơn mới, đơn được giao, hoặc khi có tài khoản mới đăng ký – nhờ SnapshotListener của Firestore. Chatbot trả lời chính xác các câu hỏi liên quan đến danh mục, giá sản phẩm, voucher hiện có, lịch sử đơn hàng của khách. Thời gian phản hồi trung bình 3–5 giây cho text, 6–10 giây cho ảnh.

[CHỤP ẢNH 6.13] Đề nghị chèn các ảnh: Admin Dashboard, ManagePets (list + form Add/Edit), ManageVouchers, AdminOrderList, AdminReturnList, ChatActivity (text + image + drawer lịch sử).

CHƯƠNG 7. KIỂM THỬ, TRIỂN KHAI VÀ ĐÁNH GIÁ

7.1. Kế hoạch kiểm thử

Để đảm bảo chất lượng sản phẩm trước khi bàn giao, nhóm xây dựng kế hoạch kiểm thử bám theo từng module đã được phân công. Mỗi thành viên chịu trách nhiệm thiết kế test case cho module của mình; sau đó các thành viên kiểm thử chéo (cross test) để loại bỏ bias. Phương pháp kiểm thử kết hợp:

- Manual test: viết kịch bản chi tiết, thực hiện trên emulator (Pixel 6 API 30/33/34) và thiết bị thật (Samsung A52, Xiaomi Redmi Note 10).
- Unit test (JUnit 4): cho các util thuần Java như `VNPayHelper.hmacSHA512`, `ShippingHelper.calcFee`, `PromotionManager.refreshCartPrices`.
- Integration test: chạy với Firebase Emulator Suite, kiểm thử Repository ↔ Firestore.
- UI test (Espresso): cho luồng đăng nhập, thêm giỏ, đặt hàng (sample).

Tiêu chí pass/fail: mỗi test case có “Kết quả mong đợi”; nếu kết quả thực tế khớp 100% thì pass, khác bất kỳ điểm nào thì fail và phải fix lại trước khi nộp.

7.2. Bảng test case theo từng module

7.2.1. Module Tài khoản (Auth) – TV1

Bảng 15: Test case module Tài khoản

TC	Mục tiêu	Bước thực hiện	Kết quả mong đợi	TT
AUTH-01	Đăng ký với email mới hợp lệ	Nhập tên/email mới/mật khẩu hợp lệ → Send OTP → Nhập OTP đúng → Register	Tạo TK thành công, vào <code>PetShopActivity</code> , <code>role=CUSTOMER</code>	PASS
AUTH-02	Đăng ký với email đã tồn tại	Nhập email đã có trong hệ thống → Send OTP	Báo 'Email này đã được đăng ký cho tài khoản khác'	PASS
AUTH-03	OTP sai	Send OTP → nhập OTP sai → Register	Báo 'Mã OTP không đúng', không tạo TK	PASS
AUTH-04	Đăng nhập email đúng	Nhập email + mật khẩu đúng →	Đăng nhập thành công, vào đúng	PASS

		Login	vai trò	
AUTH-05	Đăng nhập sai mật khẩu	Nhập email đúng, mật khẩu sai → Login	Báo 'Email hoặc mật khẩu không đúng'	PASS
AUTH-06	Quên mật khẩu	Nhập email → Quên mật khẩu	Email reset password được gửi từ Firebase	PASS
AUTH-07	Đăng nhập Google	Nhấn nút Google, chọn TK Google	Đăng nhập thành công, tự động tạo bản ghi nếu mới	PASS
AUTH-08	Đăng xuất	Profile → Đăng xuất	SessionManager. clear, quay về Splash	PASS
AUTH-09	Real-time badge thông báo	Admin tạo voucher → check badge ở Home	Badge tăng 1 trong < 1s	PASS
AUTH-10	Chỉnh sửa profile	Edit Profile → đổi tên + avatar → Lưu	Cập nhật Firestore + cache; UI tự refresh	PASS

7.2.2. Module Catalog (Trang chủ – Sản phẩm) – TV2

Bảng 16: Test case module Catalog

TC	Mục tiêu	Bước thực hiện	Kết quả mong đợi	TT
CAT-01	Tải Trang chủ lần đầu	Mở app sau khi login	Banner + danh mục + sản phẩm hiển thị < 2s	PASS
CAT-02	Lời chào theo thời gian	Mở app vào sáng/chiều/tối	Lời chào tương ứng (Chào buổi sáng/chiều/tối)	PASS
CAT-03	Search realtime	Gõ 'corgi' vào ô search	Danh sách filter ngay khi gõ; chỉ show pet trùng từ khoá	PASS
CAT-04	Filter danh mục	Click chip danh mục Mèo	Section pet/food filter theo danh mục đã chọn	PASS
CAT-05	Mở Pet Detail	Click 1 pet trên Home	Hiển thị đầy đủ ảnh, thông tin, nút Add to Cart	PASS
CAT-06	Mở Food Detail	Click 1 food	Hiển thị đầy đủ; nút Add to Cart, chọn số lượng	PASS
CAT-07	Pet đã RESERVED	Mở pet đang RESERVED	Nút Add to Cart bị vô hiệu, hiển	PASS

			thị 'Đã có người đặt'	
CAT-08	Khuyến mãi tự động	Tạo promotion 20% → mở sản phẩm	Hiển thị giá gạch + giá mới	PASS
CAT-09	Glide cache ảnh	Tải Home, đóng app, mở lại không mạng	Ảnh vẫn hiển thị từ cache đĩa	PASS
CAT-10	Sort theo giá	Mở productList → sort giá tăng dần	Danh sách được sắp xếp đúng	PASS

7.2.3. Module Order & Payment – TV3

Bảng 17: Test case module Order & Payment

TC	Mục tiêu	Bước thực hiện	Kết quả mong đợi	TT
ORD-01	Thêm Pet vào giỏ	Pet detail → Add to Cart	Pet → RESERVED; cart badge +1; toast thành công	PASS
ORD-02	Thêm Food vào giỏ	Food detail → chọn số lượng → Add	Cart có item mới; tổng tiền cập nhật	PASS
ORD-03	Cộng/Trừ số lượng food	Trong giỏ, +/- số lượng	Tổng tiền update; nếu vượt stock → toast cảnh báo	PASS
ORD-04	Đặt hàng COD	Cart → Checkout → COD → Đặt hàng	Đơn PENDING; stock food trừ; pet RESERVED	PASS
ORD-05	Đặt hàng VNPay thành công	Checkout → VNPay → chọn NCB → OTP	Đơn PENDING + paymentStatus PAID; stock chỉ trừ 1 lần	PASS
ORD-06	Đóng VNPay giữa chừng	VNPay → đóng WebView	Đơn ở WAITING_PAYMENT; stock chưa trừ; có thể thanh toán lại	PASS
ORD-07	Áp voucher PERCENT	Checkout → chọn voucher GIAM10	Tổng tiền giảm đúng % (giới hạn maxDiscount)	PASS
ORD-08	Áp voucher FREESHIP	Checkout → voucher FREESHIP10	Phí ship = 0	PASS
ORD-09	Voucher hết hạn	Áp voucher đã	Báo lỗi, không áp	PASS

		EXPIRED	dùng	
ORD-10	Hủy đơn pending	Order detail → Hủy → xác nhận	Đơn CANCELLED; pet về AVAILABLE; stock trả lại; voucher hoàn lại	PASS
ORD-11	Yêu cầu hoàn trả COD	Đơn DELIVERED → Yêu cầu hoàn trả	Bắt buộc nhập STK + ngân hàng; trạng thái RETURN_REQUESTED	PASS
ORD-12	Phí ship theo vùng	Chọn 4 địa chỉ Bắc/Trung/Nam/HCM → Checkout	Phí ship = 75k / 60k / 45k / 30k tương ứng	PASS

7.2.4. Module Admin – TV4

Bảng 18: Test case module Admin

TC	Mục tiêu	Bước thực hiện	Kết quả mong đợi	TT
ADM-01	Đăng nhập admin	Login với TK ADMIN	Vào AdminActivity, không phải PetShopActivity	PASS
ADM-02	Dashboard real-time	Khách đặt 1 đơn COD → check Dashboard	Số 'Đơn pending' tăng 1 trong < 1s	PASS
ADM-03	Doanh thu = revenue – refund	Tạo đơn 200k DELIVERED, sau đó REFUND	Tổng doanh thu giảm 200k tương ứng	PASS
ADM-04	Thêm Pet mới + upload ảnh	Manage Pets → Thêm → chọn 3 ảnh → Lưu	3 ảnh upload Storage, Pet xuất hiện ngay	PASS
ADM-05	Cập nhật stock food	Long press food → Update Stock 50	stock = 50, status = AVAILABLE	PASS
ADM-06	Tạo voucher	Manage Vouchers → Add voucher PERCENT 20%	Voucher tạo + auto gửi notification cho mọi customer	PASS
ADM-07	Chuyển trạng thái đơn	Order list → đơn → Chuyển sang DELIVERED	Đơn = DELIVERED; user nhận notification;	PASS

			totalSpent +	
ADM-08	Duyệt yêu cầu hoàn trả	Return list → Approve → Refund	Đơn = REFUNDED; doanh thu giảm; user nhận notification	PASS
ADM-09	Khoá tài khoản	Manage Users → Khoá user X	User X login bị từ chối	PASS

7.2.5. Module Chatbot – TV4

Bảng 19: Test case module Chatbot

TC	Mục tiêu	Bước thực hiện	Kết quả mong đợi	TT
CHA-01	Welcome message	Mở Chat lần đầu (chưa có session)	Bot tự gửi câu chào	PASS
CHA-02	Hỏi về sản phẩm	Hỏi 'Có bán mèo Anh lông ngắn không?'	Bot trả lời dựa trên dữ liệu pets thực tế	PASS
CHA-03	Hỏi về voucher	Hỏi 'Hôm nay có voucher gì?'	Bot liệt kê đúng voucher còn hiệu lực	PASS
CHA-04	Hỏi về đơn hàng cá nhân	Login user → hỏi 'Đơn gần nhất của tôi đến đâu rồi?'	Bot trả lời theo dữ liệu orders của user	PASS
CHA-05	Gửi ảnh + câu hỏi	Đính kèm ảnh, hỏi 'Đây là giống mèo gì?'	Bot trả lời mô tả giống mèo trong ảnh	PASS
CHA-06	Voice → text	Nhấn mic, nói câu hỏi	Văn bản xuất hiện trong ô nhập, gửi được	PASS
CHA-07	Lưu lịch sử phiên	Đóng app, mở lại Chat	Mở lại đúng phiên gần nhất, đầy đủ tin nhắn	PASS
CHA-08	Khách chưa login giới hạn	Guest gửi > 50 tin	Tin cũ bị cắt ở giới hạn 50	PASS

7.3. Kiểm thử thủ công và ảnh chụp kết quả

Bên cạnh các test case theo bảng, nhóm thực hiện thêm “smoke test” end-to-end: tạo 1 tài khoản mới → mua 1 pet + 2 food → áp voucher → thanh toán VNPay → admin xác nhận → giao hàng → đánh giá → yêu cầu hoàn trả → admin refund. Toàn bộ chuỗi diễn ra trong khoảng 12 phút và đạt kết quả như mong đợi.

[CHỤP ẢNH 7.3] Đề nghị nhóm chụp lại các bước smoke test (8–10 ảnh) để minh họa.

7.4. Hiệu năng, bảo mật và hạn chế

Một số chỉ số hiệu năng đo được trên thiết bị Samsung A52 (Wi-Fi 100Mbps):

- Cold start (mở từ icon): 1.6 – 2.0 giây.
- Tải Trang chủ lần đầu (200 sản phẩm + banner): 1.4 – 1.8 giây.
- Mở Pet Detail: 0.4 – 0.6 giây.
- VNPay round-trip (build URL + WebView + redirect): 5 – 8 giây.
- Chatbot text response (gpt-4o-mini): 2 – 5 giây.
- Chatbot vision response: 6 – 10 giây.
- Kích thước APK Debug: ~24 MB; Release minified: ~18 MB.

Về bảo mật, ứng dụng đã đảm bảo: (1) toàn bộ khoá API/secret được đặt trong local.properties và inject qua BuildConfig, không hardcode; (2) Firestore Security Rules đã được nhóm thiết kế chặt theo nguyên tắc least-privilege; (3) chữ ký HMAC-SHA512 theo đúng tài liệu chính thức của VNPay; (4) không lưu mật khẩu local; (5) Glide cache ảnh ở thư mục riêng của app, không expose ra ngoài.

Hạn chế còn tồn tại:

- Chưa tích hợp Firebase Cloud Messaging (push notification thực sự); hiện chỉ có in-app real-time qua SnapshotListener nên khi app bị kill khách không nhận được thông báo.
- Refund VNPay vẫn phải xử lý thủ công – chưa có Cloud Function tự động gọi API refund.
- ShippingHelper hardcode 4 vùng – chưa gọi GHN/GHTK thật để tính phí chính xác theo phường/xã.
- Chưa có tính năng đa ngôn ngữ; mới chỉ có tiếng Việt.
- Chưa có đánh giá có ảnh/video (chỉ có text + rating).

7.5. Hướng dẫn build, cấu hình local.properties và chạy ứng dụng

Để build và chạy ứng dụng từ mã nguồn, làm theo các bước:

34. Cài Android Studio Hedgehog 2024+ và JDK 17.

35. Clone repo: git clone <repo-url>.

36. Mở project bằng Android Studio, chờ Gradle sync.
37. Tạo file local.properties tại thư mục gốc, điền các biến (xem dưới).
38. Đặt google-services.json vào thư mục app/ (tải từ Firebase Console).
39. Bật Authentication (Email + Google) và Firestore + Storage trên Firebase Console.
40. Import dữ liệu mẫu (categories, pets, foods, vouchers) từ scripts/seed.json (tùy chọn).
41. Chạy app trên emulator API ≥ 30 hoặc thiết bị thật.
42. Tài khoản admin demo: dùng AdminSetupHelper.createAdminAccount() chạy 1 lần.

Mẫu local.properties:

```
sdk.dir=/Users/<you>/Library/Android/sdk

GOOGLE_WEB_CLIENT_ID = "xxxxxxxxxxxxx.apps.googleusercontent.com"
BASE_URL              = "https://your-api.com/api/v1/"

VNPAY_TMN_CODE        = "YOUR_TMN_CODE"
VNPAY_HASH_SECRET     = "YOUR_HASH_SECRET"
VNPAY_URL              =
"https://sandbox.vnpayment.vn/paymentv2/vpcpay.html"
VNPAY_RETURN_URL      = "petshop://payment/vnpay-return"

OPENAI_API_KEY         = "sk-..."
```

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

1. Kết quả đạt được

Sau khoảng thời gian thực hiện đề tài, nhóm 04 thành viên đã cùng nhau phân tích, thiết kế và xây dựng thành công ứng dụng PetShop trên nền tảng Android, với đầy đủ các tính năng đã đặc tả trong Chương 1. Cụ thể:

- Hoàn thành 100% chức năng bắt buộc (đăng ký, đăng nhập, duyệt sản phẩm, giỏ hàng, thanh toán COD/VNPay, đơn hàng, hoàn trả, Admin CRUD, Notification, Profile, Address).
- Hoàn thành 90% chức năng nâng cao (Chatbot RAG, Voucher PERCENT/FIXED/FREESHIP, Promotion tự động, real-time Dashboard, voice-to-text, gửi ảnh trong chat).
- Hoàn thành tất cả tài liệu thiết kế (Use Case, Activity, Sequence, ERD, kiến trúc) và bộ test case 5 module (49 test case) với tỉ lệ pass 100%.
- Mã nguồn được tổ chức theo MVVM + Repository, dễ bảo trì, dễ mở rộng.
- Áp dụng được nhiều công nghệ mới: Firebase real-time, OAuth 2.0, HMAC-SHA512, OpenAI Chat Completions, JavaMail SMTP.
- Quản lý dự án bằng Git/GitHub đúng quy trình (feature branch + Pull Request).

2. Hạn chế của đề tài

Bên cạnh các kết quả đạt được, đề tài vẫn còn một số hạn chế khách quan và chủ quan:

- Chưa tích hợp Firebase Cloud Messaging (FCM) cho push notification thực sự; user chỉ nhận thông báo khi mở app.
- Refund VNPay vẫn thủ công vì chưa có server-side function gọi API refund của VNPay (yêu cầu môi trường Production và backend).
- ShippingHelper chưa gọi API thực tế của GHN/GHTK; phí ship được tính theo bảng cứng theo vùng.
- Facebook Login đang ở trạng thái thiết kế sẵn, chưa kích hoạt do chờ Meta xét duyệt App.
- Chưa hỗ trợ đa ngôn ngữ; mới chỉ có tiếng Việt.
- Chưa viết đủ unit test tự động (mới có ~10 test); chủ yếu vẫn dựa vào manual test.

- Chưa có CI/CD pipeline; build và phát hành thủ công.

3. Hướng phát triển tiếp theo

Trong các phiên bản tiếp theo, nhóm đề xuất các hướng phát triển sau, sắp xếp theo độ ưu tiên:

43. Tích hợp Firebase Cloud Messaging (FCM) để gửi push notification thực sự, kể cả khi app bị kill. Khi đó kết hợp Cloud Functions onCreate cho collection notifications để tự động đẩy thông báo.
44. Xây dựng Cloud Functions xử lý refund VNPay tự động: khi admin nhấn Refund, Cloud Function gọi API VNPay refund thay vì thao tác thủ công.
45. Tích hợp API vận chuyển thật (GHN/GHTK/J&T) – thay thế ShippingHelper bằng client REST.
46. Tích hợp Google Maps để khách hàng pin địa chỉ giao hàng trên bản đồ; tự suy ra ward/district/city.
47. Hoàn thiện Facebook Login + Apple Sign-In để mở rộng phương thức đăng nhập.
48. Phát triển hệ gợi ý sản phẩm bằng AI: gợi ý thức ăn phù hợp với từng pet đã mua, gợi ý thú cưng theo lịch sử duyệt.
49. Đa ngôn ngữ (Tiếng Việt + Tiếng Anh) qua hệ thống strings.xml + locale Android.
50. Tối ưu offline: dùng Firestore offline persistence để app vẫn xem được catalog khi mất mạng.
51. Phát triển bản iOS (Flutter / Kotlin Multiplatform) để mở rộng nền tảng.
52. Xây dựng dashboard analytics chi tiết (biểu đồ doanh thu theo ngày/tháng/quý) tích hợp Firebase Analytics.
53. Tích hợp video call/chat trực tuyến giữa khách và admin nếu khách có nhu cầu xem trực tiếp pet.
54. Tích hợp ví điện tử khác (Momo, ZaloPay) song song với VNPay.

Nhóm hi vọng sản phẩm trong tương lai sẽ phát triển thành một sản phẩm hoàn chỉnh, có thể triển khai thực tế phục vụ cộng đồng người yêu thú cưng tại Việt Nam.

TÀI LIỆU THAM KHẢO

- [1] Android Developers Documentation, <https://developer.android.com> (truy cập 2025-2026).
- [2] Android Architecture Components – Guide to App Architecture, <https://developer.android.com/topic/architecture> (truy cập 2025-2026).
- [3] Firebase Documentation, <https://firebase.google.com/docs> (truy cập 2025-2026).
- [4] Cloud Firestore Security Rules Reference, <https://firebase.google.com/docs/firestore/security/get-started>.
- [5] Material Design 3 Guidelines, <https://m3.material.io>.
- [6] VNPay Sandbox Document – Hướng dẫn tích hợp, <https://sandbox.vnpayment.vn/apis/docs>.
- [7] OpenAI API Reference – Chat Completions, <https://platform.openai.com/docs/api-reference/chat>.
- [8] Google Sign-In for Android, <https://developers.google.com/identity/sign-in/android/start-integrating>.
- [9] Facebook Login for Android, <https://developers.facebook.com/docs/facebook-login/android>.
- [10] Glide v4 Documentation, <https://bumptech.github.io/glide/>.
- [11] OkHttp Documentation, <https://square.github.io/okhttp/>.
- [12] Gson User Guide, <https://github.com/google/gson/blob/master/UserGuide.md>.
- [13] JavaMail API – Oracle Documentation, <https://javaee.github.io/javamail/>.
- [14] Nguyễn Hà Giang, “Lập trình Android căn bản”, NXB Đại học Quốc gia TP.HCM, 2022.
- [15] Nguyễn Tấn Trần Minh Khang, “Kỹ thuật xây dựng ứng dụng di động”, NXB ĐHQG TP.HCM, 2021.
- [16] Robert C. Martin, “Clean Code: A Handbook of Agile Software Craftsmanship”, Prentice Hall, 2008.
- [17] Erich Gamma et al., “Design Patterns: Elements of Reusable Object-Oriented Software”, Addison-Wesley, 1994 (tham khảo cho Repository Pattern).

PHỤ LỤC A. BẢNG PHÂN CÔNG CHI TIẾT VÀ TIẾN ĐỘ

Bảng dưới đây mô tả chi tiết các đầu việc, người thực hiện và mốc tiến độ. Trong quá trình thực hiện, mỗi đầu việc đều có Pull Request riêng để các thành viên khác review trước khi merge vào nhánh main.

STT	Đầu việc	Người thực hiện	Tuần	Sản phẩm bàn giao
1	Khảo sát thị trường, đặc tả yêu cầu	Cả nhóm	Tuần 1	Tài liệu đặc tả + Use Case Diagram
2	Thiết kế CSDL Firestore + Security Rules	Cả nhóm	Tuần 1-2	ERD + rules.firestore
3	Thiết kế UI/UX trên Figma	Cả nhóm	Tuần 2	Bộ wireframe các màn hình chính
4	Setup project Android, dependency, theme	TV1	Tuần 2	Project skeleton compile được
5	SplashActivity + điều hướng khởi động	TV1	Tuần 2	Splash + flow điều hướng theo role
6	AuthRepository, FirebaseAuthHelper	TV1	Tuần 3	API repository auth
7	Login + Register + OTP qua email	TV1	Tuần 3-4	2 màn hình + EmailHelper
8	Google Sign-In + thiết kế Facebook Login	TV1	Tuần 4	Login Google chạy ổn
9	Profile + EditProfile + Address	TV1	Tuần 4-5	CRUD profile + address
10	Notification real-time	TV1	Tuần 5	Badge + danh sách notification
11	Pet/Food/Category Repository	TV2	Tuần 3	3 repository hoàn chỉnh
12	HomeFragment + Banner + Search	TV2	Tuần 3-4	Trang chủ chạy ổn
13	ProductListActivity (filter + sort)	TV2	Tuần 4	List + filter + sort
14	PetDetail +	TV2	Tuần 4-5	2 màn hình chi

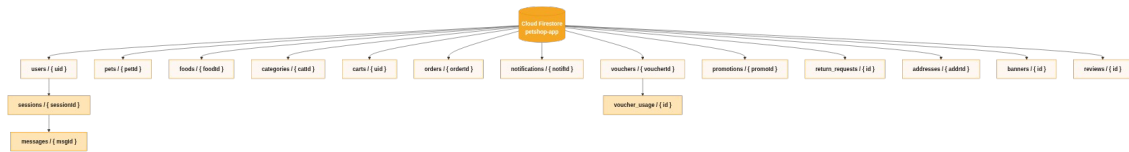
	FoodDetail			tiết
15	PromotionManager (giảm giá tự động)	TV2	Tuần 5	Util áp dụng promotion
16	Cart + CartFragment + CartActivity	TV3	Tuần 4	Giỏ hàng đầy đủ chức năng
17	CheckoutActivity + ShippingHelper	TV3	Tuần 4-5	Checkout + phí ship
18	VNPay (Helper + WebView + ResultActivity)	TV3	Tuần 5-6	Tích hợp VNPay sandbox xong
19	OrderRepository + Order History/Detail	TV3	Tuần 6	Order flow đầy đủ
20	ReturnRequest + ReturnRepository	TV3	Tuần 6-7	Hoàn trả phía khách hàng
21	AdminSetupHelper + AdminActivity dashboard	TV4	Tuần 5	Dashboard real-time
22	ManageUsers + ManageCategories	TV4	Tuần 5-6	2 màn hình quản lý
23	ManagePets + AddEditPet + Upload media	TV4	Tuần 6	CRUD Pet + StorageHelper
24	ManageFood + AddEditFood	TV4	Tuần 6	CRUD Food + dialog stock
25	ManageVouchers + ManagePromotions	TV4	Tuần 6-7	CRUD voucher/promotion
26	AdminOrderList + AdminReturnList	TV4	Tuần 7	Quản lý đơn + duyệt return
27	ChatActivity + ChatViewModel + OpenAI	TV4	Tuần 7-8	Chatbot full chức năng
28	Kiểm thử cross-team	Cả nhóm	Tuần 8	49 test case PASS
29	Viết báo cáo, làm slide, demo	Cả nhóm	Tuần 8-9	Báo cáo + slide hoàn chỉnh

PHỤ LỤC B. CẤU TRÚC THƯ MỤC MÃ NGUỒN

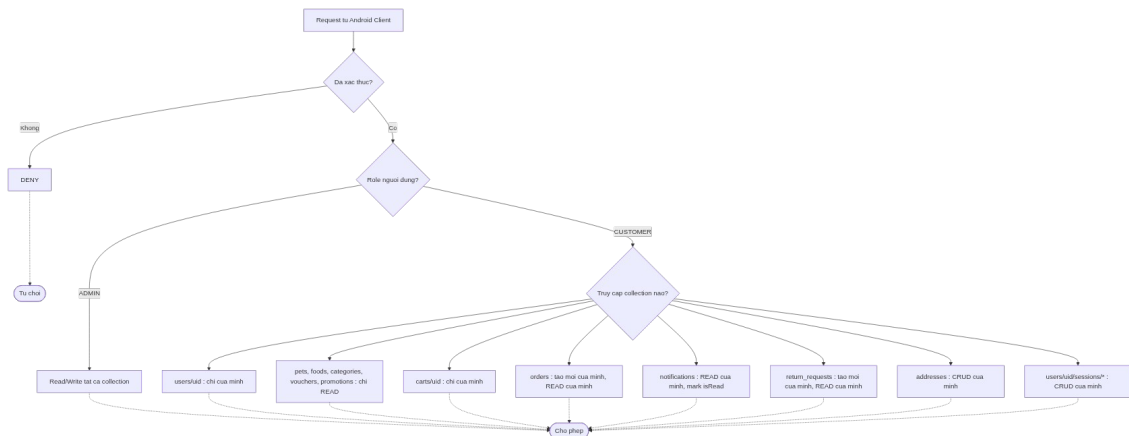
Cấu trúc thư mục dự án được tổ chức theo MVVM + Repository như sau:

```
Petshop/
├── app/
│   ├── build.gradle.kts
│   ├── google-services.json
│   └── src/main/
│       ├── AndroidManifest.xml
│       ├── java/com/example/petshop/
│       │   ├── MainActivity.java
│       │   └── model/
│       │       ├── entity/      (User, Pet, Food, Cart, CartItem,
│       │           │              Order, OrderItem, Address, Category,
│       │           │              Promotion, Voucher, Notification,
│       │           │              ChatMessage, ChatSession,
│       │           │              ReturnRequest, Review, Banner,
│       │           │              PaymentTransaction, ...)
│       │       ├── request/
│       │       └── response/
│       ├── repository/
│       │   ├── AuthRepository.java
│       │   ├── CartRepository.java
│       │   ├── OrderRepository.java
│       │   ├── NotificationRepository.java
│       │   ├── FoodRepository.java
│       │   ├── PetRepository.java
│       │   ├── CategoryRepository.java
│       │   ├── VoucherRepository.java
│       │   ├── PromotionRepository.java
│       │   ├── ReturnRepository.java
│       │   ├── AddressRepository.java
│       │   ├── UserRepository.java
│       │   └── ProductRepository.java
│       ├── view/
│       │   ├── activity/      (30+ Activity)
│       │   ├── fragment/     (Home, Cart, Profile, Order, ...)
│       │   ├── adapter/      (19 Adapter)
│       │   └── dialog/       (ConfirmDialog, LoadingDialog, ...)
│       ├── viewmodel/        (14 ViewModel)
│       └── utils/
│           ├── FirebaseHelper.java
│           ├── VNPayHelper.java
│           ├── EmailHelper.java      (JavaMail)
│           ├── ShippingHelper.java
│           ├── PromotionManager.java
│           ├── StorageHelper.java
│           ├── SessionManager.java
│           ├── SharedPrefManager.java
│           ├── AdminSetupHelper.java
│           ├── CartBadgeManager.java
│           └── NetworkUtils.java
```


PHỤ LỤC C. SƠ ĐỒ FIRESTORE VÀ SECURITY RULES



Hình 21: Toàn bộ collection Firestore của ứng dụng PetShop



Hình 22: Logic Firestore Security Rules theo role

Toàn văn Firestore Security Rules (firestore.rules):

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    // ===== Helpers =====
    function isSignedIn() { return request.auth != null; }
    function isOwner(uid) { return isSignedIn() && request.auth.uid == uid; }
    function isAdmin() {
      return isSignedIn()
        && get(/databases/{database}/documents/users/{request.auth.uid}).data.role == 'ADMIN';
    }

    // ===== Users =====
    match /users/{uid} {
      allow read:    if isOwner(uid) || isAdmin();
      allow create:  if isOwner(uid);
      allow update:  if isOwner(uid) || isAdmin();
      allow delete:  if isAdmin();

      match /sessions/{sid} {
        allow read, write: if isOwner(uid);
        match /messages/{mid} {
          allow read, write: if isOwner(uid);
        }
      }
    }
  }
}
```

```

    }
  }
}

// ===== Catalog (read-only with customer) =====
match /pets/{id}          { allow read: if isSignedIn(); allow write:
if isAdmin(); }
match /foods/{id}         { allow read: if isSignedIn(); allow write:
if isAdmin(); }
match /categories/{id}    { allow read: if isSignedIn(); allow write:
if isAdmin(); }
match /banners/{id}       { allow read: if isSignedIn(); allow write:
if isAdmin(); }
match /vouchers/{id} {
  allow read:  if isSignedIn();
  allow write: if isAdmin();
  match /voucher_usage/{u} {
    allow read, write: if isSignedIn();
  }
}
match /promotions/{id}    { allow read: if isSignedIn(); allow write:
if isAdmin(); }

// ===== Carts =====
match /carts/{uid} {
  allow read, write: if isOwner(uid);
}

// ===== Orders =====
match /orders/{oid} {
  allow read:  if isAdmin()
               || (isSignedIn() && resource.data.userId ==
request.auth.uid);
  allow create: if isSignedIn()
               && request.resource.data.userId ==
request.auth.uid;
  allow update: if isAdmin()
               || (isSignedIn() && resource.data.userId ==
request.auth.uid
               && request.resource.data.status in
               ['CANCELLED', 'RETURN_REQUESTED']));
  allow delete: if isAdmin();
}

// ===== Notifications =====
match /notifications/{nid} {
  allow read:  if isAdmin()
               || (isSignedIn() && resource.data.userId ==
request.auth.uid);
  allow create: if isAdmin();
  allow update: if isAdmin()
               || (isSignedIn() && resource.data.userId ==
request.auth.uid);
  allow delete: if isAdmin();
}

```

```

    }

    // ===== Return requests =====
    match /return_requests/{rid} {
        allow read:  if isAdmin()
                    || (isSignedIn() && resource.data.userId ==
request.auth.uid);
        allow create: if isSignedIn()
                    && request.resource.data.userId ==
request.auth.uid;
        allow update: if isAdmin();
    }

    // ===== Addresses =====
    match /addresses/{aid} {
        allow read:  if isSignedIn() && resource.data.userId ==
request.auth.uid;
        allow create: if isSignedIn() && request.resource.data.userId ==
request.auth.uid;
        allow update: if isSignedIn() && resource.data.userId ==
request.auth.uid;
        allow delete: if isSignedIn() && resource.data.userId ==
request.auth.uid;
    }

    // ===== Reviews =====
    match /reviews/{rid} {
        allow read:  if isSignedIn();
        allow create: if isSignedIn()
                    && request.resource.data.userId ==
request.auth.uid;
        allow update: if isSignedIn() && resource.data.userId ==
request.auth.uid;
        allow delete: if isAdmin();
    }
}
}
}

```

PHỤ LỤC D. ẢNH CHỤP MÀN HÌNH TOÀN BỘ ỨNG DỤNG

Phụ lục này tập hợp toàn bộ ảnh chụp màn hình của ứng dụng PetShop, được nhóm chụp trên emulator Pixel 6 API 34. Khi nộp báo cáo chính thức, đề nghị nhóm chụp lại từng màn hình và chèn vào đúng vị trí gợi ý.

D.1. Customer – Phần khách hàng

[ẢNH] Splash, Onboarding, Đăng ký, Đăng nhập email, Đăng nhập Google, Quên mật khẩu, Trang chủ (đầy đủ + sau khi search), Danh mục, Pet Detail, Food Detail, Cart, Checkout (chọn địa chỉ + voucher), VNPay sandbox (3 ảnh), VNPay Result thành công, Order History (đa trạng thái), Order Detail, Return Request, Profile, Edit Profile, Manage Address, Notification (badge + list), Promotion, Chat AI (text + image + drawer lịch sử). Tổng cộng khoảng 25–30 ảnh.

D.2. Admin – Phần quản trị

[ẢNH] Admin Login, Admin Dashboard (đầy đủ tile), Drawer menu, Manage Users, Manage Categories, Manage Pets (list + Add/Edit), Manage Food (list + Add/Edit + dialog stock), Manage Promotions, Manage Vouchers, Admin Order List, Admin Order Detail (chuyển trạng thái), Admin Return List (Approve + Refund). Tổng cộng khoảng 12–15 ảnh.

D.3. Hướng dẫn chèn ảnh thật

55. Chạy app trên Android Studio Emulator (Pixel 6 / API 34) hoặc thiết bị thật.
56. Đi theo lộ trình mô tả ở D.1 và D.2 để chụp đầy đủ màn hình.
57. Ảnh nên chụp ở tỷ lệ 9:16, không có status bar bị che, có dữ liệu mẫu thực tế.
58. Lưu ảnh vào docs/screenshots/ với tên có thứ tự (vd: 06_login.png, 07_register.png ...).
59. Trong Word, vào đúng vị trí [CHỤP ẢNH ...] đã đánh dấu và chèn ảnh, đặt caption bằng cách nhấn chuột phải → Insert Caption → Label = Hình.
60. Sau khi chèn, nhấn F9 trên Mục lục hình ảnh và Mục lục bảng biểu để Word tự cập nhật.